



AX 外围设备驱动开发指南

文档版本: V1.6

发布日期: 2024/04/30

AEXRA CONFIDENTIAL FOR SIPEED

目录

前 言	9
修订历史	10
1 pinmux 操作指南	11
1.1 操作准备	11
1.2 操作过程	13
1.2.1 uboot pinmux 配置	13
1.2.2 kernel pinmux 配置	15
1.3 各组 pinmux 驱动能力	15
1.3.1 G2 组室温/高温	15
1.3.2 G1,G6, G8,G9,G11,G12 组 1.8V 室温	17
1.3.3 G1,G6, G8,G9,G11,G12 组 1.8V 高温	18
1.3.4 G1,G6, G8,G9,G11,G12 组 3.3V 室温	19
1.3.5 G1,G6, G8,G9,G11,G12 组 3.3V 高温	20
1.4 PAD 电压切换	21
1.4.1 操作示例	21
1.5 Sleep mode	24
1.5.1 检查版本是否支持 pinmux sleep, 有如下两种方法	24
1.5.2 配置 pinmux sleep mode	24
2 GPIO 操作指南	26
2.1 操作准备	26
2.2 操作过程	26
2.2.1 uboot 操作 GPIO	28
2.2.2 通过 Linux 命令操作 GPIO	29
2.2.3 内核态 GPIO 读写操作	29

2.2.4 用户态 GPIO 操作程序示例	31
2.3 需要注意的问题	31
3 APB SPI 操作指南	34
3.1 操作准备	34
3.2 配置 pinmux	34
3.3 控制器驱动在 kernel 中使能	34
3.4 内核设备树的配置	35
4 AHB SPI SLV 操作指南	38
4.1 操作准备	38
4.2 配置 pinmux	38
4.3 控制器驱动在 kernel 中使能	38
4.4 内核设备树的配置	39
5 DDR 操作指南	41
5.1 操作准备	41
5.2 操作过程	41
5.2.1 DDR 数据率配置	41
5.2.2 DDR LPDDR4(x)/DDR4 配置	41
5.2.3 DDR 调试信息输出	42
5.3 操作实例	42
5.4 需要注意的问题	42
6 OTP 操作指南	44
操作准备	45
6.1 操作过程	45
6.2 操作示例	45
6.3 需要注意的问题	45

7 外设 DMA 操作指南	47
7.1 操作准备	47
7.2 操作过程	47
8 UART 操作指南	48
8.1 操作准备	48
8.2 需要注意的问题	49
9 EMMC 操作指南	50
9.1 操作准备	50
9.2 需要注意的问题	50
10 带宽统计操作指南	52
10.1 操作准备	52
10.2 操作过程	52
10.2.1 查看带宽	52
10.2.2 设置统计周期	53
10.2.3 查看统计周期	53
10.2.4 记录带宽到文件	53
10.3 需要注意的问题	54
11 Audio Codec 操作指南	56
11.1 内置 codec 驱动操作	56
11.1.1 添加内置 codec driver	56
11.1.2 修改 DTS	57
11.1.3 编译 axp 包	60
11.2 外置 codec 驱动操作	60
11.2.1 操作准备	60
11.2.2 添加外置 codec driver	60

11.2.3 修改 DTS	60
11.2.4 编译 axp 包	62
11.3 内外置 codec 混用驱动配置	62
11.3.1 操作准备	62
11.3.2 添加外置 codec driver	63
11.3.3 添加内置 codec driver	63
11.3.4 修改 DTS	63
11.3.5 编译 axp 包	66
11.4 需要注意的问题	66
12 USB 操作指南	70
12.1 操作准备	70
12.2 操作过程	70
12.3 操作实例	71
12.4 需要注意的问题	74
13 EMAC 操作指南	75
13.1 操作准备	75
13.2 操作过程	75
13.3 需要注意的问题	78
14 I2C 操作指南	81
14.1 操作准备	81
14.2 操作准备	81
14.2.1 配置 pinmux	81
14.2.2 控制器驱动在 kernel 中的命令	81
14.2.3 内核设备树的配置	82
14.3 I2C Recovery	83
15 RTC 操作指南	85

15.1 操作准备	85
15.2 时间同步与 NTP	85
15.3 rtc sample 测试	85
15.4 需要注意的问题	85
16 PWM 外设操作指南	86
16.1 操作准备	86
16.1.1 AX pwm 介绍	86
16.1.2 配置 pinmux	87
16.1.3 配置 u-boot	87
16.1.4 配置 kernel	88
16.2 操作实例	88
16.2.1 pinmux 配置	88
16.2.2 u-boot 中使用	89
16.2.3 应用层使用	91
16.2.4 kernel 中使用	91
17 Thermal 操作指南	93
17.1 操作准备	93
17.2 查看当前温度	95
18 NPU 信息操作指南	96
18.1 NPU 利用率	96
18.2 NPU 模型耗时信息	97
18.3 NPU 版本信息	98
19 UID 操作指南	99
19.1 操作准备	99
19.2 需要注意的问题	99

20 SD/SDIO 操作指南	100
20.1 操作准备	100
20.2 操作过程	100
20.2.1 SD 卡	100
20.2.2 SDIO 接口	103
20.2.3 SDIO WiFi 外设	104
20.3 需要注意的问题	106
21 ADC 操作指南	107
21.1 操作准备	107
21.2 操作过程	107
22 APB TIMER 操作指南	109
22.1 操作准备	109
22.2 操作过程	109
22.3 需要注意的问题	110
23 PINCTRL 操作指南	111
23.1 操作准备	111
23.2 操作过程	112
23.3 需要注意的问题	113
24 DDR 带宽限制操作指南	116
24.1 操作准备	116
24.2 操作过程	116
24.3 需要注意的问题	116
25 I2C SLV 操作指南	117
25.1 操作准备	117
25.2 操作示例	117

25.3 需要注意的问题.....	117
-------------------	-----

AEXRA CONFIDENTIAL FOR SIPEED

权利声明

爱芯元智半导体股份有限公司或其许可人保留一切权利。

非经权利人书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

注意

您购买的产品、服务或特性等应受商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非商业合同另有约定，本公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

前言

本文档介绍 AX620E 外围设备驱动开发相关的内容，本文章目前共分为 26 个章节来介绍各模块常见疑问，旨在为您在开发过程中提供帮助。希望在后续的使用过程中能够为您提供帮助，也希望可以提出更多建议或意见：

适用产品

AX620E 系列产品（AX630C、AX620Q）

适读人群

- 软件开发工程师
- 技术支持工程师

符号与格式定义

符号/格式	说明
<code>xxx</code>	表示您可以执行的命令行。
<i>斜体</i>	表示变量。如，“安装目录/AX620E_SDK_Vx.x.x/build 目录”中的“安装目录”是一个变量，由您的实际环境决定。
 说明/备注：	表示您在使用产品的过程中，我们向您说明的事项。
 注意：	表示您在使用产品的过程中，需要您特别注意的事项。

修订历史

文档版本	发布时间	修订说明
V1.0	2023/08/30	文档初版
V1.1	2023/09/13	pinmux 增加 default func 列，同时生成简化 pinmux 配置文件
V1.2	2023/10/27	dma 增加 kernel 同步接口
V1.3	2023/10/30	增加 SDIO WiFi 说明
V1.3	2023/12/25	增加 watchdog 说明
V1.4	2024/01/19	更新 Audio Codec 操作指南
V1.5	2024/03/04	增加快启项目 i2c 使用注意
V1.6	2024/04/30	增加 pin 休眠支持，章节 1.5
V1.7	2024/07/15	增加 Audio Codec 操作指南中内外置 codec 混用配置

1 pinmux 操作指南

1.1 操作准备

Pinmux 位于 sdk 工程的路径为：build/tools/pinmux/。该路径下包含 SDK 所支持的工程对应的 pinmux 配置文件，如：

```

|   |—— pinmux
|   |   |—— AX620Q_DEMO_pinmux.xlsm
|   |   |—— AX620Q_EVB_pinmux.xlsm

```

上面的 pinmux 表格分别对应不同型号芯片的 pinmux 配置。

pinmux 的配置步骤在 pinmux/readme.txt 文件中有说明如下：

How to use this pinmux tool?

step 1. open xxx_pinmux.xlsm and enable “启用编辑”

step 2. select pad function and pull selection in drop-down box

step 3. click EXPORT box to create pinmux.h in current directory

下面我们以 ax620Q_EVB_pinmux.xlsm 文件为例，根据 readme.txt 文件中的说明，来操作生成对应的 pinmux.h 文件。

步骤1 打开 ax620Q_EVB_pinmux.xlsm 文件,首先点击启用内容，如下图位置

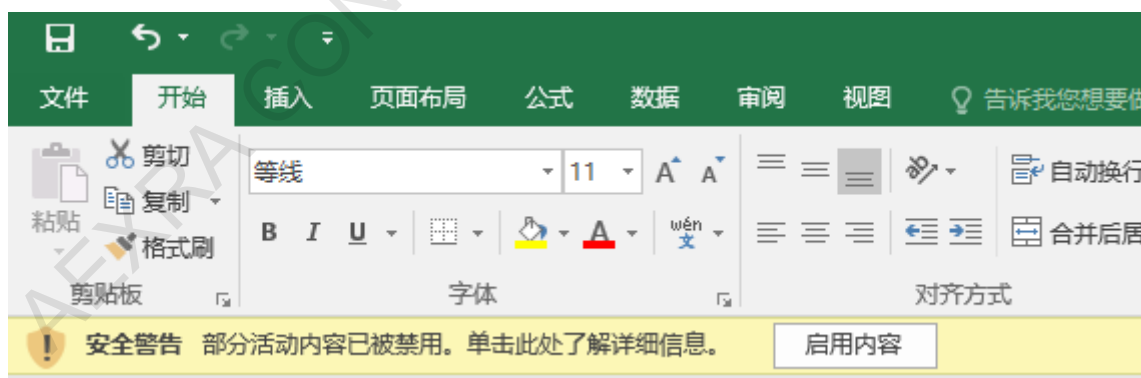


图1-1 启用内容

点击“启用内容”按钮，当弹出如下时：

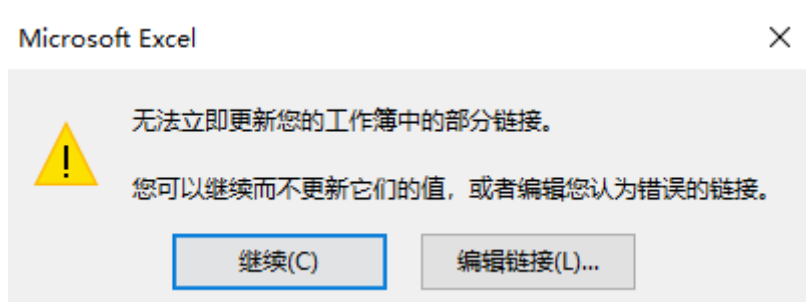


图1-2 继续

点击继续，文件进入可编辑状态。

根据表头，可知每一列对应的功能

4	group ID	PAD name	base address	offset	sleep	sleep	sleep	sleep	sleep	sleep	sleep	func_sel[18:16]	function	pu[7]	pd[6]	st[4]	ds[3:0]
1	group ID	PAD name	base address	offset	sleep_ps[2]	sleep_pe[2]	sleep_st[2]	sleep_ie[2]	sleep_oe[2]	sleep_out[2]	sleep_en[2]	func_sel[18:16]	function	ps[7]	pe[6]	st[4]	ds[3:0]

图1-3 功能表

(备注： st[4]列：施密特触发器使能； ds[3:0]列：驱动能力，详细信息见[章节 1.3](#)。

步骤2 上下拉配置需要区分 pu, pd 和 ps, pe, 区别如下：

pu[7]: pull up; pd[6]: pull down

pe[6]: pull enable, high active; ps[7]: pull selection; 0 pulldown; 1 pullup;

pu 和 pd 列表直接对应上下拉，pe 和 ps 方式配置上下拉时，需要先配置 pe 使能，再配置 ps，选择上下拉。

步骤3 根据 pad function，从下拉列表中选择正确的配置项

group ID	PAD name	base address	offset	func_sel[18:16]	function	pu[7]	pd[6]	st[4]	ds[3:0]
G1	VI_D0	0x2300000	0xC	4	I2S0_DIN0	0	0	0	3
	VI_D1		0x18		VI_D0	0	0	0	3
	VI_D2		0x24		INFRARED_FS	0	0	0	3
	VI_D3		0x30		SPI_M1_MOSI	0	0	0	3
	VI_D4		0x3C		I2C6_SDA	0	0	0	3
	VI_CLK0		0x84		I2S0_DIN0	0	0	0	3
	I2C0_SCL		0xC		DB_GPIO0	1	0	0	3
	I2C0_SDA		0x18		GPIO0_A0	1	0	0	3
			0x18		ANALOG_TEST1	1	0	0	3

图1-4 Func_sel 表

例如：根据 function 所选择的项 I2S0_DIN0，产生对应的 func_sel 值 4

步骤4 在完成配置后，点击 DEMO 页的“EXPORT”按钮（图 1-5），将配置信息导出到

xx_pinmux.h 和 xx_pinmux_simplify.h(简化后的 pinmux 配置文件, 不包含所选 func 和芯片 default func 相同的 pin)。default func 见表格 params 中最右列 (图 1-6)。

	A	B	C	D	E	F	G
1		Board ID = 5 for 620Q DEMO					
2							
3	Export						
4	group ID	PAD name	base address	offset	sleep_pu[29]	sleep_pd[28]	sleep_st[27]
5		VI_D0		0xC			
6		VI_D1		0x18			

图1-5 Export

PAD name	FUNC0	FUNC1	FUNC2	FUNC3	FUNC4	FUNC5	FUNC6	FUNC7	DEFAULT FUNC
vi_d0	VI_D0	INFRARED_FS	SPI_M1_MOSI	I2C6_SDA	I2S0_DIN0	DB_GPIO0	GPIO0_A0	ANALOG_TEST1	GPIO0_A0
vi_d1	VI_D1	INFRARED_SDO0	SPI_M1_MISO	I2C6_SCL	I2S0_SCLK	DB_GPIO1	GPIO0_A1	ANALOG_TEST2	GPIO0_A1
vi_d2	VI_D2	INFRARED_SDO1	SPI_M1_CS0	TIME_50HZ	I2S0_DOUT	DB_GPIO2	GPIO0_A2	ANALOG_TEST3	GPIO0_A2
vi_d3	VI_D3	INFRARED_SDO2	SPI_M1_CS1	CLK_AUX1_1	I2S0_MCLK	DB_GPIO3	GPIO0_A3	ANALOG_TEST4	GPIO0_A3

图1-6 Default Func

在 excel 同级目录下会生成 xx_pinmux.h 文件, 在本例中使用时可重命名 ax620Q_EVB_pinmux.h。

其中打开 ax620Q_EVB_pinmux.h 文件可以看出:

```
0x0230000C, 0x00040003, /* PadName = VI_D0          Fuction = I2S0_DIN0 */
0x0230000C: 表示寄存器地址
0x00040003: 表示通过设置表产生对应的值, 对应 32 位寄存器的相应位。
```

1.2 操作过程

Pinmux 有两处配置的地方, uboot 和 kernel 都会配置 pinmux, 使用同一份 pinmux.h 文件, 该文件放在 build\projects\AX620E_xxx\pinmux 目录下

1.2.1 uboot pinmux 配置

步骤1 首先根据实际硬件版本 (AX620E_xxx) 在文件夹 u-boot-2020.04\board\axera 下新建 AX620E_xxx 工程目录。

步骤2 在该文件夹下增加 pinmux.c 文件, 可以拷贝其他工程目录下的 pinmux.c 文件, 查看目前 pinmux.c 所列举的代码是否满足需求: 我们以 ax620Q_EVB_pinmux 为例;

```
unsigned int ax620Q_EVB_pinmux [] = {
#include "ax620Q_EVB_pinmux.h"
};
```

```
static struct pinmux ax620E_pinmux_tbl[AX620E_BOARD_MAX] = {
    [AX620Q_LP4_EVB_V1_0] =
        {ax620Q_EVB_pinmux,
         sizeof(ax620Q_EVB_pinmux) / sizeof(unsigned int)},
    .....
}
```

步骤3 在 `pinmux.c` 文件中，注意 `index = get_board_id()` 函数，该 `index` 值用来区分相应的芯片，如果是其他方式区分或者不区分需要客制化，如前面介绍的如下：

	A	B	C	D	E	F	G
1		Board ID = 5 for 620Q DEMO					
2							
	Export						
3							
4	group ID	PAD name	base address	offset	sleep_pu[29]	sleep_pd[28]	sleep_st[27]
5		VI_D0		0xC			
6		VI_D1		0x18			

图1-7 Board id

```
//根据项目客制化，此例是根据 Board ID 来区分使用不同的 xxx_pinmux.h 进行 pinmux 的初始化
int pinmux_init(void)
{
    int i;
    int index = get_board_id(); //获取板子 id 号
    u8 is_dphytx;

    if (index < 0 || index > AX620E_BOARD_MAX - 1)
        return 0;

    index = ax_pinmux_index_conv(index); //部分板子 pinmux 相同，id 号不同，进行
    id 转换，减少 pinmux 表格填充

    for (i = 0; i < ax620E_pinmux_tbl[index].size; i += 2) {
        .....
        writel(ax620E_pinmux_tbl[index].data[i + 1],
               (long)ax620E_pinmux_tbl[index].data[i]);
        .....
    }
    .....
}
```

总结：选中 `xxx_pinmux.h` 头文件通过写寄存器直接在 `uboot` 中初始化。

1.2.2 kernel pinmux 配置

步骤 1 确保 build 目录下已经存在 xxx_pinmux.h 文件。

步骤 2 在 kernel\linux\linux-5.15.73(linux-4.19.125)\drivers\soc\axera\pinmux\ax_pinmux.c 文件中，修改如下两个数组，下文中的[AX620Q_EVB]中的 AX620Q_EVB 值为 board id，需要根据自己的硬件适配。

```
unsigned int ax620Q_EVB_pinmux [] = {
#include "ax620Q_EVB_pinmux.h"
};

struct pinmux ax620E_pinmux_tbl[AX620E_BOARD_MAX] = {
    [AX620Q_LP4_EVB_V1_0] =
        {AX620Q_EVB_pinmux,
        sizeof(AX620Q_EVB_pinmux) / sizeof(unsigned int)},
};
```

1.3 各组 pinmux 驱动能力

Pin 的驱动能力受环境温度及电压不同会有所不同，下面会列出室温(25℃)和高温(85℃)及不同电压情况下的各组 pin 的驱动能力。

其中 G2,G5,G7 组只有 1.8V 电压配置，G1,G6, G8,G9,G11,G12 组有 1.8V 和 3.3V 两种电压配置。

1.3.1 G2 组室温/高温

INPUT		Output PAD					
DR[1]	DR[0]	Driving Capability(VDDIO=1.8V)					
		IOH @ VOH=0.8*VDDIO=1.44V			IOL @ VOL=0.2*VDDIO=0.36V		
		Worst	Typical	Best	Worst	Typical	Best
		SS/125℃	TT/25℃	FF/-40℃	SS/125℃	TT/25℃	FF/-40℃
0	0	3.77mA	5.36mA	7.05mA	3.45mA	4.53mA	5.53mA
0	1	7.53mA	10.72mA	14.11mA	6.93mA	9.05mA	11.07mA
1	0	15.07mA	21.45mA	28.22mA	13.87mA	18.11mA	22.14mA
1	1	26.38mA	37.54mA	49.38mA	24.27mA	31.69mA	38.74mA

G5,G7 组室温/高温

Parameter		Min.	Nom.	Max.	Units
V_{OL}	Output Low Voltage			0.45	V
V_{OH}	Output High Voltage	VDDIO-0.45			V
I_{OL}	Low Level Output Current @ $V_{OL}(\max)$				
	(DS3,DS2,DS1,DS0) = '0000'	3.8	5.5	7.3	mA
	(DS3,DS2,DS1,DS0) = '0001'	7.2	10.6	14.1	mA
	(DS3,DS2,DS1,DS0) = '0010'	10.4	15.3	20.5	mA
	(DS3,DS2,DS1,DS0) = '0011'	13.2	19.6	26.2	mA
	(DS3,DS2,DS1,DS0) = '0100'	16.3	24.2	32.4	mA
	(DS3,DS2,DS1,DS0) = '0101'	18.6	27.7	37.2	mA
	(DS3,DS2,DS1,DS0) = '0110'	20.7	30.8	41.5	mA
	(DS3,DS2,DS1,DS0) = '0111'	22.6	33.8	45.5	mA
	(DS3,DS2,DS1,DS0) = '1000'	25.3	37.9	51.0	mA
	(DS3,DS2,DS1,DS0) = '1001'	26.8	40.1	54.0	mA
	(DS3,DS2,DS1,DS0) = '1010'	28.1	42.1	56.8	mA
	(DS3,DS2,DS1,DS0) = '1011'	29.4	43.9	59.2	mA
	(DS3,DS2,DS1,DS0) = '1100'	30.6	45.8	61.5	mA
	(DS3,DS2,DS1,DS0) = '1101'	31.7	47.4	63.7	mA
	(DS3,DS2,DS1,DS0) = '1110'	32.7	48.8	65.7	mA
	(DS3,DS2,DS1,DS0) = '1111'	33.6	50.2	67.6	mA
I_{OH}	High Level Output Current @ $V_{OH}(\min)$				
	(DS3,DS2,DS1,DS0) = '0000'	3.2	4.8	6.6	mA
	(DS3,DS2,DS1,DS0) = '0001'	6.1	9.2	12.6	mA
	(DS3,DS2,DS1,DS0) = '0010'	8.7	13.3	18.4	mA
	(DS3,DS2,DS1,DS0) = '0011'	11.1	17.0	23.5	mA
	(DS3,DS2,DS1,DS0) = '0100'	13.6	21.0	29.0	mA
	(DS3,DS2,DS1,DS0) = '0101'	15.5	24.1	33.3	mA
	(DS3,DS2,DS1,DS0) = '0110'	17.2	26.8	37.2	mA
	(DS3,DS2,DS1,DS0) = '0111'	18.8	29.3	40.7	mA
	(DS3,DS2,DS1,DS0) = '1000'	21.1	32.9	45.8	mA
	(DS3,DS2,DS1,DS0) = '1001'	22.3	34.8	48.4	mA
	(DS3,DS2,DS1,DS0) = '1010'	23.4	36.6	50.9	mA
	(DS3,DS2,DS1,DS0) = '1011'	24.5	38.2	53.1	mA
	(DS3,DS2,DS1,DS0) = '1100'	25.6	39.8	55.3	mA
	(DS3,DS2,DS1,DS0) = '1101'	26.4	41.2	57.2	mA
	(DS3,DS2,DS1,DS0) = '1110'	27.2	42.4	59.0	mA
	(DS3,DS2,DS1,DS0) = '1111'	28.0	43.6	60.6	mA

1.3.2 G1,G6, G8,G9,G11,G12 组 1.8V 室温

IOL/IOH	Min.	This_PVT	Max.	Units
IOL Low Level Output Current @ V_{OL} (max)				
(DS3,DS2,DS1,DS0) = '0000'	1.9	2.6	4.1	mA
(DS3,DS2,DS1,DS0) = '0001'	3.7	5.0	7.9	mA
(DS3,DS2,DS1,DS0) = '0010'	5.7	7.7	12.2	mA
(DS3,DS2,DS1,DS0) = '0011'	7.3	9.7	15.4	mA
(DS3,DS2,DS1,DS0) = '0100'	9.3	12.4	19.8	mA
(DS3,DS2,DS1,DS0) = '0101'	11.1	14.8	23.5	mA
(DS3,DS2,DS1,DS0) = '0110'	12.9	17.1	27.1	mA
(DS3,DS2,DS1,DS0) = '0111'	14.4	19.2	30.3	mA
(DS3,DS2,DS1,DS0) = '1000'	21.6	28.7	45.3	mA
(DS3,DS2,DS1,DS0) = '1001'	23.4	31.1	48.9	mA
(DS3,DS2,DS1,DS0) = '1010'	25.4	33.7	53.1	mA
(DS3,DS2,DS1,DS0) = '1011'	26.9	35.7	56.2	mA
(DS3,DS2,DS1,DS0) = '1100'	28.6	38.0	59.8	mA
(DS3,DS2,DS1,DS0) = '1101'	30.4	40.3	63.3	mA
(DS3,DS2,DS1,DS0) = '1110'	32.1	42.6	66.9	mA
(DS3,DS2,DS1,DS0) = '1111'	33.6	44.6	69.9	mA
IOH High Level Output Current @ V_{OH} (max)				
(DS3,DS2,DS1,DS0) = '0000'	1.8	2.5	4.1	mA
(DS3,DS2,DS1,DS0) = '0001'	3.5	4.8	7.9	mA
(DS3,DS2,DS1,DS0) = '0010'	5.5	7.4	12.3	mA
(DS3,DS2,DS1,DS0) = '0011'	6.9	9.3	15.5	mA
(DS3,DS2,DS1,DS0) = '0100'	8.9	11.9	19.9	mA
(DS3,DS2,DS1,DS0) = '0101'	10.6	14.2	23.7	mA
(DS3,DS2,DS1,DS0) = '0110'	12.2	16.5	27.4	mA
(DS3,DS2,DS1,DS0) = '0111'	13.7	18.5	30.6	mA
(DS3,DS2,DS1,DS0) = '1000'	20.8	28.0	46.6	mA
(DS3,DS2,DS1,DS0) = '1001'	22.5	30.2	50.4	mA
(DS3,DS2,DS1,DS0) = '1010'	24.4	32.8	54.7	mA
(DS3,DS2,DS1,DS0) = '1011'	25.9	34.8	57.9	mA
(DS3,DS2,DS1,DS0) = '1100'	27.6	37.0	61.7	mA
(DS3,DS2,DS1,DS0) = '1101'	29.2	39.3	65.4	mA
(DS3,DS2,DS1,DS0) = '1110'	30.9	41.6	69.1	mA
(DS3,DS2,DS1,DS0) = '1111'	32.4	43.5	72.3	mA

1.3.3 G1,G6, G8,G9,G11,G12 组 1.8V 高温

IOL/IOH	Min.	This_PVT	Max.	Units
IOL Low Level Output Current @ V_{OL} (max)				
(DS3,DS2,DS1,DS0) = '0000'	1.9	3.0	4.1	mA
(DS3,DS2,DS1,DS0) = '0001'	3.7	5.8	7.9	mA
(DS3,DS2,DS1,DS0) = '0010'	5.7	8.9	12.2	mA
(DS3,DS2,DS1,DS0) = '0011'	7.3	11.3	15.4	mA
(DS3,DS2,DS1,DS0) = '0100'	9.3	14.4	19.8	mA
(DS3,DS2,DS1,DS0) = '0101'	11.1	17.2	23.5	mA
(DS3,DS2,DS1,DS0) = '0110'	12.9	19.9	27.1	mA
(DS3,DS2,DS1,DS0) = '0111'	14.4	22.2	30.3	mA
(DS3,DS2,DS1,DS0) = '1000'	21.6	33.2	45.3	mA
(DS3,DS2,DS1,DS0) = '1001'	23.4	35.9	48.9	mA
(DS3,DS2,DS1,DS0) = '1010'	25.4	39.0	53.1	mA
(DS3,DS2,DS1,DS0) = '1011'	26.9	41.3	56.2	mA
(DS3,DS2,DS1,DS0) = '1100'	28.6	44.0	59.8	mA
(DS3,DS2,DS1,DS0) = '1101'	30.4	46.6	63.3	mA
(DS3,DS2,DS1,DS0) = '1110'	32.1	49.3	66.9	mA
(DS3,DS2,DS1,DS0) = '1111'	33.6	51.5	69.9	mA
IOH High Level Output Current @ V_{OH} (max)				
(DS3,DS2,DS1,DS0) = '0000'	1.8	2.9	4.1	mA
(DS3,DS2,DS1,DS0) = '0001'	3.5	5.6	7.9	mA
(DS3,DS2,DS1,DS0) = '0010'	5.5	8.7	12.3	mA
(DS3,DS2,DS1,DS0) = '0011'	6.9	11.0	15.5	mA
(DS3,DS2,DS1,DS0) = '0100'	8.9	14.1	19.9	mA
(DS3,DS2,DS1,DS0) = '0101'	10.6	16.8	23.7	mA
(DS3,DS2,DS1,DS0) = '0110'	12.2	19.4	27.4	mA
(DS3,DS2,DS1,DS0) = '0111'	13.7	21.7	30.6	mA
(DS3,DS2,DS1,DS0) = '1000'	20.8	32.9	46.6	mA
(DS3,DS2,DS1,DS0) = '1001'	22.5	35.6	50.4	mA
(DS3,DS2,DS1,DS0) = '1010'	24.4	38.6	54.7	mA
(DS3,DS2,DS1,DS0) = '1011'	25.9	40.9	57.9	mA
(DS3,DS2,DS1,DS0) = '1100'	27.6	43.6	61.7	mA
(DS3,DS2,DS1,DS0) = '1101'	29.2	46.3	65.4	mA
(DS3,DS2,DS1,DS0) = '1110'	30.9	48.9	69.1	mA
(DS3,DS2,DS1,DS0) = '1111'	32.4	51.2	72.3	mA

1.3.4 G1,G6, G8,G9,G11,G12 组 3.3V 室温

IOL/IOH	Min.	This_PVT	Max.	Units
IOL Low Level Output Current @ V_{OL} (max)				
(DS3,DS2,DS1,DS0) = '0000'	1.1	1.6	2.1	mA
(DS3,DS2,DS1,DS0) = '0001'	2.1	3.1	4.1	mA
(DS3,DS2,DS1,DS0) = '0010'	3.2	4.7	6.2	mA
(DS3,DS2,DS1,DS0) = '0011'	4.2	6.2	8.2	mA
(DS3,DS2,DS1,DS0) = '0100'	5.3	7.8	10.3	mA
(DS3,DS2,DS1,DS0) = '0101'	6.4	9.3	12.3	mA
(DS3,DS2,DS1,DS0) = '0110'	7.4	10.8	14.3	mA
(DS3,DS2,DS1,DS0) = '0111'	8.4	12.2	16.2	mA
(DS3,DS2,DS1,DS0) = '1000'	12.6	18.2	24.0	mA
(DS3,DS2,DS1,DS0) = '1001'	13.6	19.7	26.0	mA
(DS3,DS2,DS1,DS0) = '1010'	14.7	21.3	28.0	mA
(DS3,DS2,DS1,DS0) = '1011'	15.7	22.7	29.9	mA
(DS3,DS2,DS1,DS0) = '1100'	16.7	24.2	31.9	mA
(DS3,DS2,DS1,DS0) = '1101'	17.7	25.7	33.8	mA
(DS3,DS2,DS1,DS0) = '1110'	18.8	27.2	35.8	mA
(DS3,DS2,DS1,DS0) = '1111'	19.8	28.6	37.6	mA
IOH High Level Output Current @ V_{OH} (max)				
(DS3,DS2,DS1,DS0) = '0000'	2.1	3.6	5.2	mA
(DS3,DS2,DS1,DS0) = '0001'	4.1	6.9	10.1	mA
(DS3,DS2,DS1,DS0) = '0010'	6.4	10.7	15.7	mA
(DS3,DS2,DS1,DS0) = '0011'	8.1	13.5	19.8	mA
(DS3,DS2,DS1,DS0) = '0100'	10.4	17.3	25.4	mA
(DS3,DS2,DS1,DS0) = '0101'	12.4	20.6	30.3	mA
(DS3,DS2,DS1,DS0) = '0110'	14.4	23.9	35.1	mA
(DS3,DS2,DS1,DS0) = '0111'	16.1	26.8	39.3	mA
(DS3,DS2,DS1,DS0) = '1000'	24.3	40.1	58.6	mA
(DS3,DS2,DS1,DS0) = '1001'	26.2	43.3	63.4	mA
(DS3,DS2,DS1,DS0) = '1010'	28.5	47.1	68.8	mA
(DS3,DS2,DS1,DS0) = '1011'	30.2	49.9	72.9	mA
(DS3,DS2,DS1,DS0) = '1100'	32.1	53.1	77.7	mA
(DS3,DS2,DS1,DS0) = '1101'	34.1	56.4	82.4	mA
(DS3,DS2,DS1,DS0) = '1110'	36.1	59.6	87.2	mA
(DS3,DS2,DS1,DS0) = '1111'	37.8	62.4	91.3	mA

1.3.5 G1,G6, G8,G9,G11,G12 组 3.3V 高温

IOL/IOH	Min.	This_PVT	Max.	Units
IOL Low Level Output Current @ $V_{OL}(\max)$				
$(DS3,DS2,DS1,DS0) = '0000'$	1.1	1.4	2.1	mA
$(DS3,DS2,DS1,DS0) = '0001'$	2.1	2.7	4.1	mA
$(DS3,DS2,DS1,DS0) = '0010'$	3.2	4.1	6.2	mA
$(DS3,DS2,DS1,DS0) = '0011'$	4.2	5.3	8.2	mA
$(DS3,DS2,DS1,DS0) = '0100'$	5.3	6.7	10.3	mA
$(DS3,DS2,DS1,DS0) = '0101'$	6.4	8.0	12.3	mA
$(DS3,DS2,DS1,DS0) = '0110'$	7.4	9.3	14.3	mA
$(DS3,DS2,DS1,DS0) = '0111'$	8.4	10.6	16.2	mA
$(DS3,DS2,DS1,DS0) = '1000'$	12.6	15.7	24.0	mA
$(DS3,DS2,DS1,DS0) = '1001'$	13.6	17.0	26.0	mA
$(DS3,DS2,DS1,DS0) = '1010'$	14.7	18.3	28.0	mA
$(DS3,DS2,DS1,DS0) = '1011'$	15.7	19.6	29.9	mA
$(DS3,DS2,DS1,DS0) = '1100'$	16.7	20.9	31.9	mA
$(DS3,DS2,DS1,DS0) = '1101'$	17.7	22.1	33.8	mA
$(DS3,DS2,DS1,DS0) = '1110'$	18.8	23.4	35.8	mA
$(DS3,DS2,DS1,DS0) = '1111'$	19.8	24.7	37.6	mA
IOH High Level Output Current @ $V_{OH}(\max)$				
$(DS3,DS2,DS1,DS0) = '0000'$	2.1	3.1	5.2	mA
$(DS3,DS2,DS1,DS0) = '0001'$	4.1	6.0	10.1	mA
$(DS3,DS2,DS1,DS0) = '0010'$	6.4	9.2	15.7	mA
$(DS3,DS2,DS1,DS0) = '0011'$	8.1	11.7	19.8	mA
$(DS3,DS2,DS1,DS0) = '0100'$	10.4	15.0	25.4	mA
$(DS3,DS2,DS1,DS0) = '0101'$	12.4	17.9	30.3	mA
$(DS3,DS2,DS1,DS0) = '0110'$	14.4	20.7	35.1	mA
$(DS3,DS2,DS1,DS0) = '0111'$	16.1	23.2	39.3	mA
$(DS3,DS2,DS1,DS0) = '1000'$	24.3	34.8	58.6	mA
$(DS3,DS2,DS1,DS0) = '1001'$	26.2	37.6	63.4	mA
$(DS3,DS2,DS1,DS0) = '1010'$	28.5	40.9	68.8	mA
$(DS3,DS2,DS1,DS0) = '1011'$	30.2	43.3	72.9	mA
$(DS3,DS2,DS1,DS0) = '1100'$	32.1	46.1	77.7	mA
$(DS3,DS2,DS1,DS0) = '1101'$	34.1	49.0	82.4	mA
$(DS3,DS2,DS1,DS0) = '1110'$	36.1	51.8	87.2	mA
$(DS3,DS2,DS1,DS0) = '1111'$	37.8	54.2	91.3	mA

1.4 PAD 电压切换

AX620E 芯片有 6 个 pad group: G1, G6, G8, G9, G11, G12 支持 pad 电压 1.8V/3.3V 切换。每个 group 有自己的 vdet enable 寄存器, G1, G6, G11 放在 G10 的 misc_ctrl 寄存器 bit[2:0]中, G8, G9, G12 放在 G8 的 misc_ctrl 寄存器 bit[2:0]中, default 1, 默认 enable, 每个 group 有自己的 vdet status(RO)寄存器, 在对应 group 的 vdet_ro 寄存器, 通过读此寄存器确认电压状态。0 表明当前检测电压为: 1.8V, 1 表明当前检测电压为 3.3V, 通过配置[ms2,ms1]选择当前 group 的电压。 [ms2, ms1] == 2'b11, 1.8V Mode; [ms2, ms1] == 2'b00, 3.3V Mode;

Group	vdet enable address default 1, default enable		vdet status address read this register, Confirm the voltage		ms2/ms1 register address	
G1	misc_ctrl_g10[0]	PINMUX_G10_BASE_ADDR + 'h1C	vdet_ro_g1[0]	PINMUX_G1_BASE_ADDR + 'h94	pin_ctrl_g1[8:7]	AON_SYS_PINMUX_G1_BASE_ADDR + 'h0
G6	misc_ctrl_g10[1]		vdet_ro_g6[0]	PINMUX_G6_BASE_ADDR + 'hA0	pin_ctrl_g6[8:7]	AON_SYS_PINMUX_G6_BASE_ADDR + 'h0
G11	misc_ctrl_g10[2]		vdet_ro_g11[0]	PINMUX_G11_BASE_ADDR + 'hA0	pin_ctrl_g11[8:7]	AON_SYS_PINMUX_G11_BASE_ADDR + 'h0
G8	misc_ctrl_g8[0]	PINMUX_G8_BASE_ADDR + 'h11C	vdet_ro_g8[0]	PINMUX_G8_BASE_ADDR + 'h118	pin_ctrl_g8[8:7]	AON_SYS_PINMUX_G8_BASE_ADDR + 'h0
G9	misc_ctrl_g8[1]		vdet_ro_g9[0]	PINMUX_G9_BASE_ADDR + 'h58	pin_ctrl_g9[8:7]	AON_SYS_PINMUX_G9_BASE_ADDR + 'h0
G12	misc_ctrl_g8[2]		vdet_ro_g12[0]	PINMUX_G12_BASE_ADDR + 'h58	pin_ctrl_g12[8:7]	AON_SYS_PINMUX_G12_BASE_ADDR + 'h0

图1-8 pin reg 说明

1.4.1 操作示例

```
Sample:
#define PIN_MUX_G1_BASE                0x02300000
#define PIN_MUX_G1_PINCTRL_SET        (PIN_MUX_G1_BASE + 0x4)
#define PIN_MUX_G1_PINCTRL_CLR        (PIN_MUX_G1_BASE + 0x8)
#define PIN_MUX_G1_VDET_RO0           (PIN_MUX_G1_BASE + 0x94)

#define PIN_MUX_G6_BASE                0x02304000
```

```

#define PIN_MUX_G6_PINCTRL_SET      (PIN_MUX_G6_BASE + 0x4)
#define PIN_MUX_G6_PINCTRL_CLR      (PIN_MUX_G6_BASE + 0x8)
#define PIN_MUX_G6_VDET_RO0         (PIN_MUX_G6_BASE + 0xA0)

#define PIN_MUX_G11_BASE             0x02309000
#define PIN_MUX_G11_PINCTRL_SET      (PIN_MUX_G11_BASE + 0x4)
#define PIN_MUX_G11_PINCTRL_CLR      (PIN_MUX_G11_BASE + 0x8)
#define PIN_MUX_G11_VDET_RO0         (PIN_MUX_G11_BASE + 0xA0)

#define PIN_MUX_G8_BASE              0x104F0000
#define PIN_MUX_G8_PINCTRL_SET        (PIN_MUX_G8_BASE + 0x4)
#define PIN_MUX_G8_PINCTRL_CLR        (PIN_MUX_G8_BASE + 0x8)
#define PIN_MUX_G8_VDET_RO0           (PIN_MUX_G8_BASE + 0x118)

#define PIN_MUX_G9_BASE              0x104F1000
#define PIN_MUX_G9_PINCTRL_SET        (PIN_MUX_G9_BASE + 0x4)
#define PIN_MUX_G9_PINCTRL_CLR        (PIN_MUX_G9_BASE + 0x8)
#define PIN_MUX_G9_VDET_RO0           (PIN_MUX_G9_BASE + 0x58)

#define PIN_MUX_G12_BASE             0x104F2000
#define PIN_MUX_G12_PINCTRL_SET        (PIN_MUX_G12_BASE + 0x4)
#define PIN_MUX_G12_PINCTRL_CLR        (PIN_MUX_G12_BASE + 0x8)
#define PIN_MUX_G12_VDET_RO0           (PIN_MUX_G12_BASE + 0x58)

/*
 * pin G1, G6, G8, G9, G11, G12 power detect step:
 * 1. check det_en in G10 msic_ctrl bit[2:0] for G1, G6, G11
 * 2. check det_en in G8 msic_ctrl bit[2:0] for G8, G9, G12
 * 3. read pad_rol reg, 1:3.3V, 0:1.8V
 * 4. cfg pin_ctrl ms1:ms2 0x3:1.8V 0x0:3.3V
 */
void pin_power_detect(void)
{
    u32 val;
    /* default vdet already enable */
    val = readl(PIN_MUX_G1_VDET_RO0);
    if (((val >> 0) & BIT(0))) { //3.3v
        writel(GENMASK(8, 7), PIN_MUX_G1_PINCTRL_CLR);
    } else { //1.8v
        writel(GENMASK(8, 7), PIN_MUX_G1_PINCTRL_SET);
    }
}

```

```
}

val = readl(PIN_MUX_G6_VDET_RO0);
if (((val >> 0) & BIT(0))) { //3.3v
    writel(GENMASK(8, 7), PIN_MUX_G6_PINCTRL_CLR);
} else { //1.8v
    writel(GENMASK(8, 7), PIN_MUX_G6_PINCTRL_SET);
}

val = readl(PIN_MUX_G11_VDET_RO0);
if (((val >> 0) & BIT(0))) { //3.3v
    writel(GENMASK(8, 7), PIN_MUX_G11_PINCTRL_CLR);
} else { //1.8v
    writel(GENMASK(8, 7), PIN_MUX_G11_PINCTRL_SET);
}

val = readl(PIN_MUX_G8_VDET_RO0);
if (((val >> 0) & BIT(0))) { //3.3v
    writel(GENMASK(8, 7), PIN_MUX_G8_PINCTRL_CLR);
} else { //1.8v
    writel(GENMASK(8, 7), PIN_MUX_G8_PINCTRL_SET);
}

val = readl(PIN_MUX_G9_VDET_RO0);
if (((val >> 0) & BIT(0))) { //3.3v
    writel(GENMASK(8, 7), PIN_MUX_G9_PINCTRL_CLR);
} else { //1.8v
    writel(GENMASK(8, 7), PIN_MUX_G9_PINCTRL_SET);
}

val = readl(PIN_MUX_G12_VDET_RO0);
if (((val >> 0) & BIT(0))) { //3.3v
    writel(GENMASK(8, 7), PIN_MUX_G12_PINCTRL_CLR);
} else { //1.8v
    writel(GENMASK(8, 7), PIN_MUX_G12_PINCTRL_SET);
}
}
```

1.5 Sleep mode

Pinmux 支持 sleep mode 配置，在系统进入 sleep mode 时，支持 pin 进入指定的状态

1.5.1 检查版本是否支持 pinmux sleep，有如下两种方法

- 0x23040A4 bit[1]是否置 1（ax_lookat 0x23040A4）
- kernel/linux/linux-xxx/drivers/soc/axera/pinmux/ax_pinmux.c 中是否有实现并引用 ax_pinmux_sleep_mode_en 函数。

1.5.2 配置 pinmux sleep mode

- sleep en: 开启 pin sleep 支持
- sleep_oe/sleep_ie: 配置 sleep mode 时，pin 是输入还是输出，sleep_oe 输出，sleep_ie 输入。
- sleep out: 配置 sleep mode 时输出高低，1 输出高，0 输出低，需要处于 sleep_oe 状态。
- sleep_st: 施密特触发器使能，同上文 st
- sleep_pu/sleep_pd/sleep_ps/sleep_pe，同上文，需要注意 sleep mode 时的驱动能力是控制一组 pin，默认驱动能力配置为 0，如需修改，需要修改文件 kernel/linux/linux-xxx/drivers/soc/axera/pinmux/ax_pinmux.c 中的宏定义，cdphy tx 和 cdphy rx 两组 pin 不支持配置休眠上下拉和驱动能力，仅支持配置输入输出状态。

```
#define SLEEP_MODE_DEFAULT_ST 0
```

具体配置如下

图 1-9 pu/pd gpio1_a0 sleep 时配置为上拉输出高，gpio1_a1 sleep 时配置为下拉输出低
gpio1_a2 sleep 时配置为输入上拉，gpio1_a3 sleep 时配置为输入下拉

group ID	pad name	base address	offset	sleep_pu[26]	sleep_pd[26]	sleep_st[27]	sleep_ie[26]	sleep_oe[26]	sleep_out[26]	sleep_in[26]	func sel[16:16]	function	pu[7]	pd[6]	st[4]	sl[3:0]	notes	explain
G6	UART2_TXD	0x2304000	0xb0	1				1		1	1	GPIO1_A0	1	0	0	2	emmc_reset_n	1 pu[7]: pull up
	UART2_RXD		0xb8		1					1	1	GPIO1_A1	1	0	0	3	asd_dat_n	2 pd[6]: pull down
	UART3_TXD		0xb4	1				1		1	1	GPIO1_A2	1	0	0	3	tx_rstn	3 pu[6]: pull enable high active
	UART3_RXD		0xb0		1			1		1	1	GPIO1_A3	1	0	0	3	gpio_exp_int_n	4 pd[7]: pull selection: 0 pulldown: 1 pullup

图1-9 pu/pd

图 1-10 ps/pe gpio3_a1 sleep 时配置为上拉输入，gpio3_a2 sleep 时配置为下拉输入

group ID	PAD name	base address	offset	sleep_ps[29]	sleep_pe[28]	sleep_st[27]	sleep_ie[26]	sleep_oe[25]	sleep_out[24]	sleep_en[23]	func_sel[18:16]	function	ps[7]	pe[6]	st[4]	ds[3:0]	notes
	GPIO3_A1		0x18	1	1		1			1	0	GPIO3_A1	1	1	0	3	sensor_RSTN0
	GPIO3_A2		0x24		1		1			1	0	GPIO3_A2	0	0	0	3	

图1-10 ps/pe

！ 注意

1. sleep mode 时的驱动能力是控制一组 pin
2. cdphy tx 和 cdphy rx 两组 pin 不支持配置休眠上下拉和驱动能力，仅支持配置输入输出状态。请参阅 1.5.2 小节第 5 点。

2 GPIO 操作指南

2.1 操作准备

首先介绍一下 AX630C 有 97 个 GPIO PIN，AX620Q 有 69 个 GPIO PIN；GPIO 一共分为四组（GPIO0-GPIO3），每组 32 个 GPIO PIN。下表列出可用 GPIO：

GPIO CHIP	AX630C
GPIO0	GPIO0_A0-GPIO0_A31
GPIO1	GPIO1_A0-GPIO1_A31
GPIO2	GPIO2_A0-GPIO2_A29
GPIO3	GPIO3_A1-GPIO3_A3

GPIO CHIP	AX620Q
GPIO0	GPIO0_A0-GPIO0_A4, GPIO0_A10-GPIO0_A31
GPIO1	GPIO1_A0-GPIO1_A7, GPIO1_A12-GPIO1_A15, GPIO1_A18-GPIO1_A21, GPIO1_A27-GPIO1_A29
GPIO2	GPIO2_A0, GPIO2_A4-GPIO2_A9, GPIO2_A10-GPIO2_A19, GPIO2_A26-GPIO2_A29
GPIO3	GPIO3_A1-GPIO3_A3

- 配置 u-boot config 文件，确保 u-boot-2020.04 中 GPIO 驱动被编译，这些配置可以写在 AX620E_emmc_defconfig 中。
- 配置 kernel config 文件，确保 kernel 中 GPIO 驱动，被编译这些配置可以写在 axera_AX620E_emmc_defconfig 中。

2.2 操作过程

以 demo 板为例，kernel config 文件位于 kernel/linux/linux-

x.xx.xx/arch/arm64/configs/axera_AX620E_emmc_defconfig，GPIO 驱动配置如下：

```
CONFIG_GPIOLIB_FASTPATH_LIMIT=512
CONFIG_OF_GPIO=y
CONFIG_GPIOLIB_IRQCHIP=y
CONFIG_GPIO_SYSFS=y
CONFIG_GPIO_GENERIC=y
CONFIG_GPIOLIB=y
CONFIG_GPIO_AXERA=y
```

kernel gpio 设备树文件位于 kernel/linux/linux-x.xx.xx/arch/arm64/boot/dts/axera/AX620E.dtsi, 配置如下:

```
ax_gpio0: gpio@4800000 {
    compatible = "axera,ax-apb-gpio";
    reg = <0x0 0x4800000 0x0 0x400>;
    resets = <&periph_reset 10 0x18 1>,
            <&periph_reset 11 0x18 1>;
    reset-names = "gpio_prst", "gpio_rst";
    clocks = <&periph_clk AX620X_CLK_GPIO0_EB>,
            <&periph_clk AX620X_PCLK_GPIO0_EB>;
    clock-names = "clk_gpio", "pclk_gpio";
    gpio-controller;
    #gpio-cells = <2>;
    interrupt-controller;
    #interrupt-cells = <2>;
    interrupts = <GIC_SPI 114 IRQ_TYPE_LEVEL_HIGH>;
    input-debounce = <1>;
    ax,ngpios = <32>;
    status = "okay";
};
```

.....

uboot config 文件位 boot/uboot/u-boot-2020.04/configs/AX620E_emmc_defconfig , GPIO 驱动配置如下:

```
# gpio #

CONFIG_AXERA_GPIO=y
CONFIG_GPIO=y
CONFIG_CMD_GPIO=y
CONFIG_DM_GPIO=y
```

uboot gpio 设备树文件位于 boot/uboot/u-boot-2020.04/arch/arm/dts/ax620e.dtsi, 配置如下:

```
ax_gpio0: gpio@4800000 {
    compatible = "axera,ax-apb-gpio";
    /* index0 is gpio reg; index1 is clk_rst_base */
    reg = <0x0 0x4800000 0x0 0x400>;
    port0a: gpio-controller@0 {
        gpio-controller;
        #gpio-cells = <2>;
```

```
ax,gpios = <32>;
bank-name = "GPIO0";
};
};
.....
```

使用 uboot driver 和 kernel driver 操作 GPIO，都需要 GPIO 统一编号，在 AX620E SOC 中 GPIO 分为#0、#1、#2、#3，每组有 32 个 GPIO，做如下约定 N 为 GPIO 的编号，P 为组号取值范围为 0、1、2、3，可以得到 $N=p*32+offset$ 。以 GPIO1_17 为例计算 GPIO 的编号，可以知道 $p=1$ ，offset 为 17， $N=1*32+17=49$

在 DTS 中需要都配置到 ax_gpio1 上，并且后面的 num 该为：17，所以配置为：

```
cs-gpios = <&ax_gpio1 17 0>;
```

2.2.1 uboot 操作 GPIO

DM 命令：1.gpio status GPIO0/ GPIO1/ GPIO2/ GPIO3，查看各个 bank 下所有已用 gpio 的状态，2.gpio input pin，将某个 pin 设置为输入，3.gpio set pin，将某个 pin 设置为输出高，4.gpio clear pin，将某个 pin 设置为输出低，5.gpio toggle pin，将某个 pin 当前输出状态进行反转。

uboot 操作 GPIO 的函数如下：

```
int ret = gpio_request(49,"test_led");//gpio 需要先申请再使用
if(ret < 0)
{
    printf("gpio request error\n");
}
ret = gpio_direction_output(49,0);
if(ret < 0)
{
    printf("gpio direction error\n");
}

int val = gpio_get_value(49);//返回值为 gpio 的状态

gpio_set_value(49,1);
```

2.2.2 通过 Linux 命令操作 GPIO

此操作示例通过命令实现对 GPIO 的读写操作。

步骤1 在控制台使用 `echo` 命令将要操作的 GPIO 编号：

```
echo N > /sys/class/gpio/export //export 之后就会生成
/sys/class/gpio/gpioN 目录
```

步骤2 在控制台使用 `echo` 命令设置 GPIO 方向：

```
echo in > /sys/class/gpio/gpioN/direction
echo out > /sys/class/gpio/gpioN/direction
```

步骤3 在控制台使用 `cat` 或 `echo` 命令查看 GPIO 输入值或设置 GPIO 输出值：

```
echo N > /sys/class/gpio/export //export 之后就会生成
/sys/class/gpio/gpioN 目录
cat /sys/class/gpio/gpioN/value //查看输入值
echo 0 > /sys/class/gpio/gpioN/value //输出低
echo 1 > /sys/class/gpio/gpioN/value //输出高
例如，设置 GPIO1_17 方向：
echo 49 > /sys/class/gpio/export
echo in > /sys/class/gpio/gpio49/direction //输入使能
cat /sys/class/gpio/gpio49/value //查看输入值
echo out > /sys/class/gpio/gpio49/direction //输出使能
echo 0 > /sys/class/gpio/gpio49/value //输出低
echo 1 > /sys/class/gpio/gpio49/value //输出高
cat /sys/class/gpio/gpio49/direction //查看 GPIO 方向 GPIO 的电平值只有 0 和
1。0 为低电平，1 为高电平。
```

步骤4 在控制台使用 `echo` 命令将操作的 GPIO 编号 `unexport`：

```
echo N > /sys/class/gpio/unexport
```

2.2.3 内核态 GPIO 读写操作

内核态 GPIO 操示例

步骤1 申请注册 GPIO：

```
gpio_request(gpio_num, NULL);
```

步骤2 设置 GPIO 方向:

```
gpio_direction_input(gpio_num) //对于输入
gpio_direction_output(gpio_num, gpio_out_val) //对于输出
```

如果是输出，需要设置一个输出的初始值，参数 `gpio_out_val` 为输出时的初始值。

步骤3 查看 GPIO 输入值或设置 GPIO 输出值:

```
gpio_get_value(gpio_num) //查看输入值
gpio_set_value(gpio_num, 0) //输出低
gpio_set_value(gpio_num, 1) //输出高
gpio_get_value(gpio_num) //返回值为输入值
```

步骤4 释放注册的 GPIO:

```
gpio_free(gpio_num);
```

内核态 GPIO 中断操作示例

步骤1 注册 GPIO:

```
gpio_request(gpio_num, NULL);
```

步骤2 设置 GPIO 方向:

对于要作为中断源的 GPIO 引脚，方向必须配置为输入。

```
gpio_direction_input(gpio_num)
```

步骤3 映射操作的 GPIO 编号对应的中断号:

```
irq_num = gpio_to_irq(gpio_num);
中断号为 gpio_to_irq(gpio_num) 的返回值
```

步骤4 注册中断:

```
request_irq(irq_num, gpio_test_isr, irqflags, "gpio_test",
&gpio_irq_type))
```

`irqflags` 为需要注册的中断类型，常用类型有:

<code>IRQF_SHARED</code>	共享中断
<code>IRQF_TRIGGER_RISING</code>	上升沿触发
<code>IRQF_TRIGGER_FALLING</code>	下降沿触发
<code>IRQF_TRIGGER_HIGH</code>	高电平触发
<code>IRQF_TRIGGER_LOW</code>	低电平触发

步骤5 结束时释放注册的中断和 GPIO 编号：

```
free_irq(gpio_to_irq(gpio_num), &gpio_irq_type);
gpio_free(gpio_num);
```

2.2.4 用户态 GPIO 操作程序示例

此操作示例在用户态下实现对 GPIO 的读写操作：

```
//将要操作的 GPIO 编号 export:
FILE *fp = fopen("/sys/class/gpio/export", "w");
fprintf(fp, "%d", gpio_num);
fclose(fp);
//设置 GPIO 方向:
FILE *fp = fopen("/sys/class/gpio/gpio%d/direction", "rb+");
fprintf(fp, "in");//对于输入
fprintf(fp, "out");//对于输出
fclose(fp);
//查看 GPIO 输入值或设置 GPIO 输出值:
FILE *fp = fopen("/sys/class/gpio/gpio%d/value", "rb+");
fread(buf, sizeof(char), sizeof(buf) - 1, fp);// 查看输入值:
strcpy(buf, "0");//设置 gpio 输出低:
fwrite(buf, sizeof(char), sizeof(buf) - 1, fp);
strcpy(buf, "1");//设置 gpio 输出高
fwrite(buf, sizeof(char), sizeof(buf) - 1, fp);
//将操作的 GPIO 编号 unexport:
fp = fopen("/sys/class/gpio/unexport", "w");
fprintf(fp, "%d", gpio_num);
fclose(fp)
```

2.3 需要注意的问题

1.需要注意的是 GPIO CONTROLLER 清中断机制改变，EOI 先写 1 再写 0 为一次清中断操作；

2.GPIO 寄存器及使用说明

AX620E 芯片设计有 4 个 gpio wrapper，每个包含 32 个 gpio，GPIO0_A[31:0]、GPIO1_A[31:0]、GPIO2_A[31:0]、GPIO3_A[31:0]，其分别对应的基地址为 GPIO0_BASE 0x4800000、

GPIO1_BASE 0x4801000、GPIO2_BASE 0x6000000、GPIO3_BASE 0x6001000。

```
//sample:

#define GPIO0_BASE 0x4800000
#define GPIO1_BASE 0x4801000
#define GPIO2_BASE 0x6000000
#define GPIO3_BASE 0x6001000

#define GPIO_SWPORTA_DR    BIT(0)
#define GPIO_SWPORTA_DDR   BIT(1)
#define GPIO_SOFT_HAR_MODE BIT(2)
#define GPIO_INTEN         BIT(3)
#define GPIO_INTMASK       BIT(4)
#define GPIO_INTTYPE_LEVEL BIT(5)
#define GPIO_INT_POLARITY  BIT(6)
#define GPIO_PORTA_DEBOUNCE BIT(7)
#define GPIO_PORTA_EOI      BIT(8)
#define GPIO_INT_BOTHEDGE   BIT(9)
#define GPIO_PORTA0_FUNC    0x4
#define GPIO_PIN_OFFSET(x) ((x + 1) * GPIO_PORTA0_FUNC)
//基地址 bit0 代表软件模式下输出数据寄存器
//基地址 bit1 代表软件模式下输入输出方向选择，0 为输入，1 为输出
//基地址 bit2 代表软件硬件模式选择，0 为软件模式，1 为硬件模式，目前暂未使用
//基地址 bit3 代表中断使能，0 为关中断，1 为开中断
//基地址 bit4 代表中断掩码，0 为关闭掩码，1 为打开掩码
//基地址 bit5 代表中断类型，0 为电平敏感，1 为边沿敏感
//基地址 bit6 代表中断极性，0 为低电平或者下降沿敏感，1 为高电平或者上升沿敏感
//基地址 bit7 代表中断消抖，0 为关闭消抖，1 为打开消抖
//基地址 bit8 代表中断清除，0 为关闭清中断，1 为打开清中断
//基地址 bit9 代表中断边沿敏感类型，0 为单边沿敏感，1 为双边沿敏感

//比如设置 GPIO2_A1 这个管脚，GPIO2 基地址是 GPIO2_BASE、然后控制这个寄存器
//no secure mode
val = readl(GPIO2_BASE);
val &= (~(1 << 1));
writel(val, GPIO2_BASE);

//direction
```

```
val = readl(GPIO2_BASE + GPIO_PIN_OFFSET(1));  
val |= GPIO_SWPORTA_DDR;  
  
writel(val, GPIO2_BASE + GPIO_PIN_OFFSET(1));  
//default high  
val = readl(GPIO2_BASE + GPIO_PIN_OFFSET(1));  
val |= GPIO_SWPORTA_DR ;  
writel(val, GPIO2_BASE + GPIO_PIN_OFFSET(1));
```

AEXRA CONFIDENTIAL FOR SIPEED

3 APB SPI 操作指南

3.1 操作准备

AX620E SOC 一共有 3 个 apb-spi 控制器，spi_mst0、spi_mst1、spi_mst2。

3.2 配置 pinmux

将相应 pin 脚的功能切换至 spi 功能。

考虑到 AX620E 系列芯片的 APB-SPI 控制器 IP 的具体实现情况，片选信号需要配置为对应的 GPIO 功能，在传输数据之前配置该 GPIO 端口输出低电平，在数据传输之后配置该 GPIO 端口输出高电平。如此操作可以保证控制器发出的命令在设备端响应的连续性。

因此，特别注意将需要用到的 cs 信号配置为相应的 GPIO 功能。

3.3 控制器驱动在 kernel 中使能

使能下图中的 DesignWare SPI controller core support 和 Memory-mapped io interface driver for DW SPI core 即可：

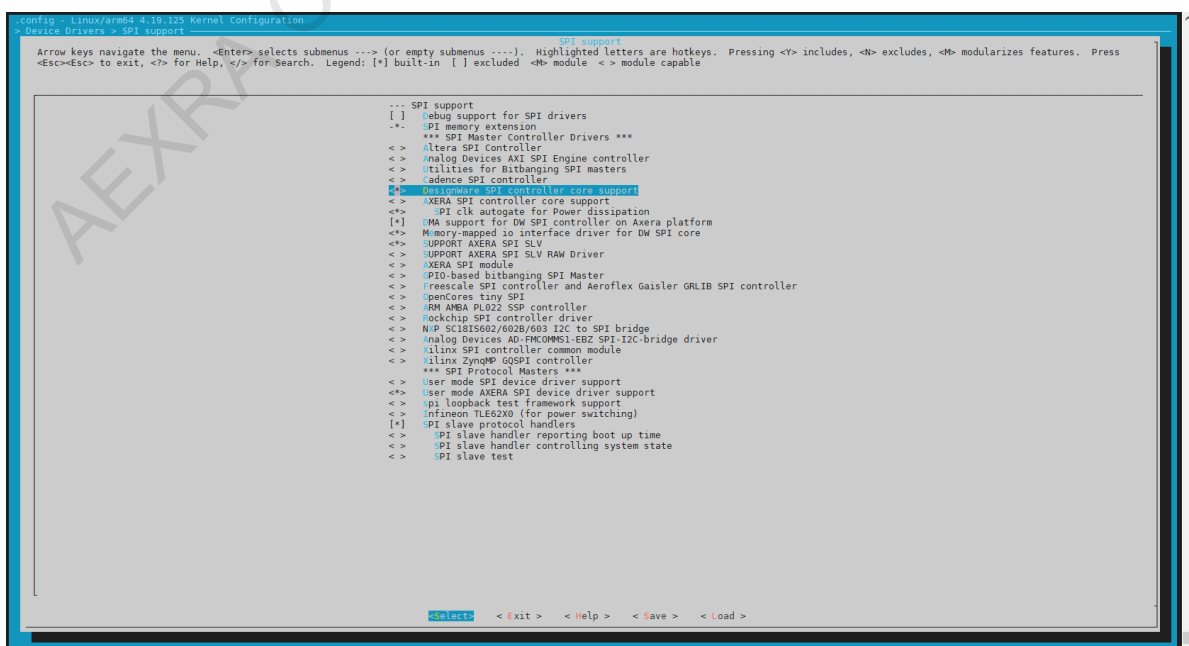


图3-1 内核配置

3.4 内核设备树的配置

在设备树配置文件 AX620E.dtsi 中有如下配置：

```
aliases {  
    ...  
    spi0 = &spi0;  
    spi1 = &spi1;  
    spi2 = &spi2;  
    ...  
}
```

spi0 的设备树配置：

```
spi0: spi@6070000 {  
    compatible = "snps,dw-apb-ssi";  
    reg = <0x0 0x6070000 0x0 0x400>;  
    interrupts = <GIC_SPI 151 IRQ_TYPE_LEVEL_HIGH>;  
    cs-gpios = <&ax_gpio2 12 0>; /*GPIO2-A12 for spi-mst0's cs, default  
select cs0 */  
    pinctrl-names = "default";  
    #address-cells = <1>;  
    #size-cells = <0>;  
    clocks = <&periph_clk AX620X_CLK_SPI_M0_EB>,  
            <&periph_clk AX620X_PCLK_SPI_M0_EB>;  
    clock-names = "apb_ssi_clk", "pclk";  
    resets = <&periph_reset 31 0x1C 1>,  
            <&periph_reset 30 0x1C 1>;  
    reset-names = "rst", "prst";  
    dmas = <&dma_per 9>, <&dma_per 8>;  
    dma-names = "rx", "tx";  
    status = "disabled";  
};
```

spi1 的设备树配置：

```
spi1: spi@6071000 {  
    compatible = "snps,dw-apb-ssi";  
    reg = <0x0 0x6071000 0x0 0x400>;
```

```

interrupts = <GIC_SPI 152 IRQ_TYPE_LEVEL_HIGH>;
cs-gpios = <&ax_gpio0 2 0>; /*GPIO0-A2 for spi-mst1's cs, default
select cs0 */
pinctrl-names = "default";
#address-cells = <1>;
#size-cells = <0>;
clocks = <&periph_clk AX620X_CLK_SPI_M1_EB>,
        <&periph_clk AX620X_PCLK_SPI_M1_EB>;
clock-names = "apb_ssi_clk", "pclk";
resets = <&periph_reset 1 0x20 1>,
        <&periph_reset 0 0x20 1>;
reset-names = "rst", "prst";
dmas = <&dma_per 11>, <&dma_per 10>;
dma-names = "rx", "tx";
status = "disabled";
};

```

spi2 的设备树配置:

```

spi2: spi@6072000 {
    compatible = "snps,dw-apb-ssi";
    reg = <0x0 0x6072000 0x0 0x400>;
    interrupts = <GIC_SPI 153 IRQ_TYPE_LEVEL_HIGH>;
    cs-gpios = <&ax_gpio0 27 0>; /*GPIO0-A27 for spi-mst2's cs, default
select cs0 */
pinctrl-names = "default";
#address-cells = <1>;
#size-cells = <0>;
clocks = <&periph_clk AX620X_CLK_SPI_M2_EB>,
        <&periph_clk AX620X_PCLK_SPI_M2_EB>;
clock-names = "apb_ssi_clk", "pclk";
resets = <&periph_reset 3 0x20 1>,
        <&periph_reset 2 0x20 1>;
reset-names = "rst", "prst";
dmas = <&dma_per 13>, <&dma_per 12>;
dma-names = "rx", "tx";
status = "disabled";
};

```

在设备树配置文件 AX630C_emmc_arm64_k419.dts 中配置如下, 就可以使用 spi0 了。

```
&spi0 {
```

```
        status = "okay";  
    };
```

AEXRA CONFIDENTIAL FOR SIPEED

4 AHB SPI SLV 操作指南

4.1 操作准备

AX630C SOC 一共就 1 个 ahb-spi-slv 控制器，spi_slv，默认我们 SDK 只支持了 1 线的 SPI 通信，4 线的暂未支持，且没有支持 DMA 传输。AX620Q SOC 不支持 spi slv。

4.2 配置 pinmux

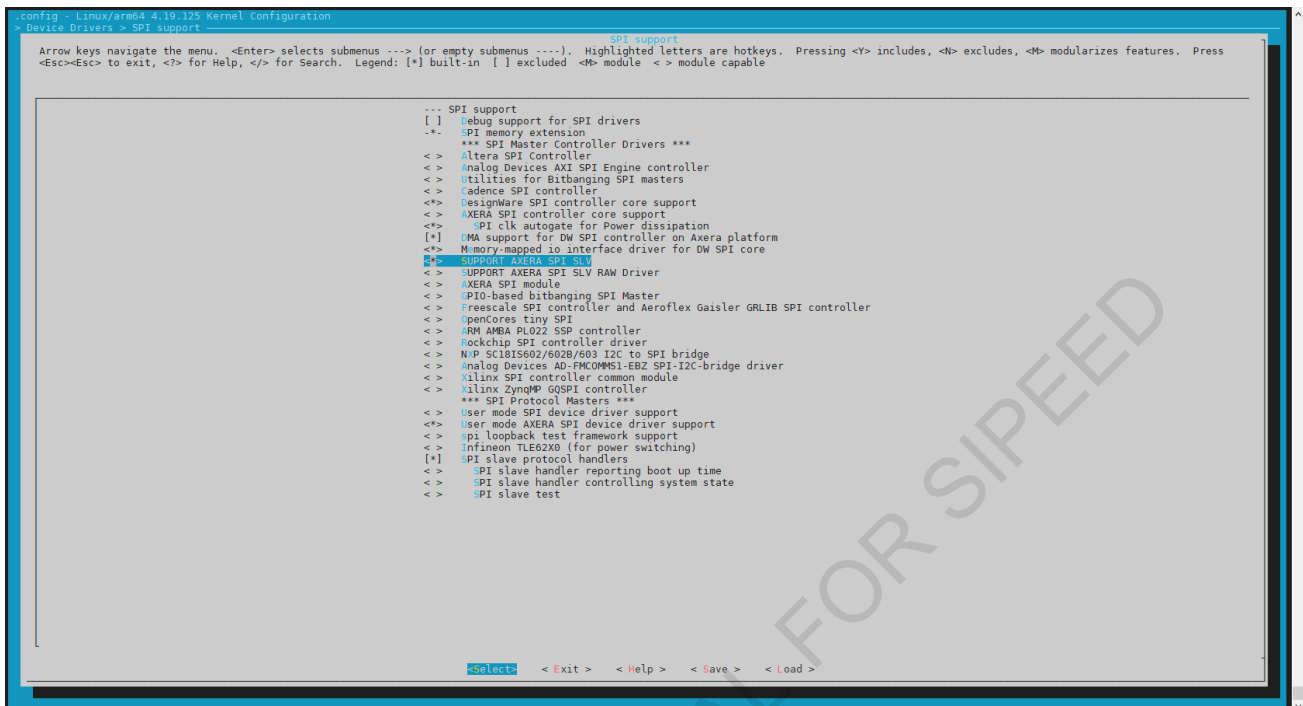
将相应 pin 脚的功能切换至 spi_s 功能。

AX630C 的 AHB-SPI-SLV 控制器 IP 需要将片选信号需要配置为 CS 功能，由 IP 控制，硬件上和 MST 的 CS 控制管脚对接。

因此，特别注意将需要用到的 CS 信号配置为 CS 功能，这是和 SPI MST 不同的地方。

4.3 控制器驱动在 kernel 中使能

在默认的 SDK 驱动中提供了两套 SPI SLV 驱动，一套是走 SPI CORE 框架的流程，一套是额外实现的 SPI SLV RAW 的驱动，使能下图中的 AXERA SPI slv raw controller core support 或 AXERA SPI slv controller core support 即可：



4.4 内核设备树的配置

在设备树配置文件 AX620E.dtsi 中有如下配置：

```

aliases {
    ...
    spi3 = &spi_slv;
    ...
}
  
```

spi0 的设备树配置：

```

spi_slv: spi@1002E000 {
    compatible = "snps,dwc-ssi-slv-1.03a";
    reg = <0x0 0x1002E000 0x0 0x2000>;
    interrupts = <GIC_SPI 88 IRQ_TYPE_LEVEL_HIGH>;
    pinctrl-names = "default";
    #address-cells = <1>;
    #size-cells = <0>;
    clocks = <&flash_clk AX620X_CLK_SPI_S_EB>,
            <&flash_clk AX620X_HCLK_SPI_S_EB>;
    clock-names = "ahb_ssi_s_clk", "hclk";
    resets = <&flash_reset 22 0x14 1>,
  
```

```

        <&flash_reset 21 0x14 1>;
    reset-names = "rst", "hrst";
    status = "disabled";
};

```

若使用 raw 的这套驱动，可以在 {demo}.dts 中添加设备使能，会生成 /dev/axera_spi_slv 节点，如下：

```

&spi_slv {
    status = "disabled";
    stream-tx-gpio = <&ax_gpio2 13 0>; //用于发送同步的 gpio，可根据实际硬件设计配置
    stream-rx-gpio = <&ax_gpio2 14 0>; //用于接收同步的 gpio，可根据实际硬件设计配置
};

```

若使用 spi core 框架的驱动，可以在 {demo}.dts 中添加设备使能，会生成 /dev/spidev3.0 节点，如下：

```

&spislrv {
    status = "okay";
    spi-slave;
    stream-ctl-enable = <1>;
    stream-rx-gpio = <&ax_gpio2 14 0>; //用于接收同步的 gpio
    slave {
        compatible = "axera,spidev";
        reg = <0>; /* config use cs0 of spi6 */
        spi-max-frequency = <25000000>;
        status = "okay";
    };
};

```

5 DDR 操作指南

5.1 操作准备

使用 AX 发布的 bl1 代码仓库。

5.2 操作过程

目前的 DDR 驱动支持的用户可对 DDR 的配置项都包含在头文件 `ddr_init_config.h`（路径：`boot/bl1/driver/ddr/ddr/ddr_init_config.h`），在编译前可通过修改宏定义来进行配置。

5.2.1 DDR 数据率配置

驱动支持的 DDR4\LPDDR4 数据率可通过以下宏定义来配置，当前 LPDDR4 支持 6 种数据率的配置，DDR4 支持 4 种数据率的配置。LPDDR4 和 DDR4 对应的宏定义都必须其中一个配置为 1，其余为 0。LPDDR4 默认使用 2400Mbps，DDR4 默认使用 2666Mbps。

```
#define LPDDR4_CONFIG_1600      0
#define LPDDR4_CONFIG_2133     0
#define LPDDR4_CONFIG_2400     1
#define LPDDR4_CONFIG_2666     0
#define LPDDR4_CONFIG_2800     0
#define LPDDR4_CONFIG_3200     0

#define DDR4_CONFIG_1600        0
#define DDR4_CONFIG_1866        0
#define DDR4_CONFIG_2133        0
#define DDR4_CONFIG_2400        0
#define DDR4_CONFIG_2666        1
```

5.2.2 DDR LPDDR4(x)/DDR4 配置

驱动支持单 RANK LPDDR4(x)或者 DDR4 的自动适配，用户不需要修改代码即可启动 DDR4 或者 LPDDR4 的板子。

AX630C 支持双 RANK LPDDR4，但由于 IRAM 大小限制可能和一些 debug 功能冲突无法共存，因而此功能默认未开启，如需使用此类规格 LPDDR4 请务必从 HW AVL list 中选择验证过的物料，软件对有此需求的项目单独维护开启 b11/drivers/ddr/ddr/ddr_common_config.h 文件下被注释的 AX630C_LPDDR4_DUAL_RANK 宏定义来支持。

5.2.3 DDR 调试信息输出

驱动支持 PHY training 信息输出，在文件 ddr_common_config.h 中打开宏 DDR_OFFLINE_SCAN

来打开 training 调试信息输出，该宏定义默认是注释掉的。将编译的 bin 烧入到板子中，重启后，会打印出 ddr ca&read&write 的眼图相关信息，ca 眼图信息只有 LPDDR4 有，DDR4 没有。

```
// #define DDR_OFFLINE_SCAN
```

这个功能的用途主要有：

1. 当用户遇到 DDR training fail 要具体分析 fail 的原因时，可以通过分析调试信息包含的 training log。
2. 当用户遇到 DDR training pass，但是系统无法启动时，也可以通过 training log 查看 ca&write&read 的眼图大小，观察眼图是否正常来判断是否是 DDR 的原因。

5.3 操作实例

SDK 目前流程是在 b11 中的 spl 进行 DDR 初始化，ROMCODE 上电启动后加载 spl 到 OCM，spl 中会调用 DDR 驱动对 DDR 进行初始化。

5.4 需要注意的问题

1、部分版本分支开启了 DDR4 初始化后 Re-Train 机制：

- ①. 机制的作用：每次上电开机 DDR 默认只做一次 training，个别物料可能随温度、负载变化参数变化较大，需要间隔周期补做 training，代价是预留出物理内存前 4KB 空间；
- ②. 机制的开启与关闭：

可查阅 build/projects/工程名/project.mak 中是否有配置 AX630C_DDR4_RETRAIN 为 TRUE 即为机制开启状态；

此机制如需关闭，可配置为 FALSE，并将 kernel dts 中对应的 reserve 空间配置删除

Gerrit CHANGES YOUR DOCUMENTATION BROWSE

86863: [AX620E-8640] linux: ddr support retrain function -- linux-4.19.125/arch/arm/boot/dts/axera/AX630C_emmc_arm32_k419.dts

Base: gitee Patchset 2: gitee DOWNLOAD

@@ +101 - Show 32 common lines - +101	@@ +101 - Show 32 common lines - +101
33 * * * reg = <ATF_RESERVED_START_HI ATF_RESERVED_START_LO ATF_RESERVED_SIZE_HI ATF_RESERVED_SIZE_LO>;	33 * * * reg = <ATF_RESERVED_START_HI ATF_RESERVED_START_LO ATF_RESERVED_SIZE_HI ATF_RESERVED_SIZE_LO>;
34 * * * no-map;	34 * * * no-map;
35 * * *};	35 * * *};
36 #ifdef OPTEE_BOOT	36 #ifdef OPTEE_BOOT
37 * * * optee_reserved: optee_memserved {	37 * * * optee_reserved: optee_memserved {
38 * * * reg = <OPTEE_RESERVED_START_HI OPTEE_RESERVED_START_LO OPTEE_RESERVED_SIZE_HI OPTEE_RESERVED_SIZE_LO>;	38 * * * reg = <OPTEE_RESERVED_START_HI OPTEE_RESERVED_START_LO OPTEE_RESERVED_SIZE_HI OPTEE_RESERVED_SIZE_LO>;
39 * * * no-map;	39 * * * no-map;
40 * * *};	40 * * *};
41 #endif	41 #endif
42 #endif	42 #endif
43 * * *};	43 * * * axera_ddr_retrain00 {
44 * * * chosen {	44 * * * compatible = "axera_ddr_retrain";
45 * * * bootargs = BOOTARGS;	45 * * * reg = <0x0 0x40000000 0x0 0x1000>;
46 * * *};	46 * * *};
47 * * *};	47 * * *};
48 * * * sound {	48 * * * chosen {
49 * * * status = "okay";	49 * * * bootargs = BOOTARGS;
50 * * * compatible = "simple-audio-card";	50 * * *};
51 * * * simple-audio-card,name = "Axera Audio";	51 * * *};
52 * * *};	52 * * * sound {
	53 * * * status = "okay";
	54 * * * compatible = "simple-audio-card";
	55 * * * simple-audio-card,name = "Axera Audio";
	56 * * *};
@@ +101 - Show 840 common lines - +101	@@ +101 - Show 840 common lines - +101

87003: [AX620E-8640] build: ddr support retrain function for build -- projects/AX630C_emmc_arm32_k419/project.mak

Base: gitee Patchset 2: gitee DOWNLOAD

@@ +101 - Show 32 common lines - +101	@@ +101 - Show 32 common lines - +101
33 AX_BOOT_OPTIMIZATION_SUPPORT := FALSE	33 AX_BOOT_OPTIMIZATION_SUPPORT := FALSE
34 SUPPORT_FIREWALL := TRUE	34 SUPPORT_FIREWALL := TRUE
35 AX_SYSDUMP_EMMC := TRUE	35 AX_SYSDUMP_EMMC := TRUE
36 #default use rsa2048	36 #default use rsa2048
37 #SIGN_USE_RSA3072 := FALSE	37 #SIGN_USE_RSA3072 := FALSE
38 COMPRESS_KO := TRUE	38 COMPRESS_KO := TRUE
39 FLASH_TYPE := emmc	39 FLASH_TYPE := emmc
40 SUPPORT_ATF := FALSE	40 SUPPORT_ATF := FALSE
41 SUPPORT_KERNEL_BOOTARGS := TRUE	41 SUPPORT_KERNEL_BOOTARGS := TRUE
42 AX_SUPPORT_AB_PART := TRUE	42 AX_SUPPORT_AB_PART := TRUE
43	43 AX630C_DDR4_RETRAIN := TRUE
44 ifeq (\$(strip \$(filter TRUE,\$(AX_SUPPORT_AB_PART) \$(SUPPORT_OPTEE) \$(SUPPORT_ATF))),TRUE)	44
45 ENABLE_SNUPDATE := TRUE	45 ifeq (\$(strip \$(filter TRUE,\$(AX_SUPPORT_AB_PART) \$(SUPPORT_OPTEE) \$(SUPPORT_ATF))),TRUE)
46 #endif	46 ENABLE_SNUPDATE := TRUE
47	47 #endif
48 ifeq (\$(strip \$(buildin_prdrv)),yes)	48
49 KERNEL_BUILDIN_PRDRV := TRUE	49 ifeq (\$(strip \$(buildin_prdrv)),yes)
50 #endif	50 KERNEL_BUILDIN_PRDRV := TRUE
51	51 #endif
52 ifeq (\$(strip \$(AX_BOOT_OPTIMIZATION_SUPPORT)),TRUE)	52
	53 ifeq (\$(strip \$(AX_BOOT_OPTIMIZATION_SUPPORT)),TRUE)
@@ +101 - Show 11 common lines - +101	@@ +101 - Show 11 common lines - +101

2、DDR4 配置为 3200 频点时 core 电压会升压到 828mv，不需要手动配置电压，DDR4 3200 频点跟 core 电压 828mv 绑定。

6 OTP 操作指南

芯片内部集成了一块 efuse，有 64 个 block，每个 block 是 32 位的存储单元。efuse(base address 0x2200000)中，blk13 为 BondOption 是用于需求设置。该 blk 的 bit 24 为 1 时，blk10-blk12、blk14、blk31-blk61 给客户使用，blk0-blk31 的 lock blk 为 62，blk32-blk61 的 lock blk 为 blk63。

该 blk 的 bit 24 为 0 时，blk0-blk30 给客户使用，blk32-blk63 给 ce 使用，blk0-blk30 的 lock blk 为 blk31，ce 没有 lock blk。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Reserved for Customer																
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved for Customer																BLK10
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Reserved for Customer																
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved for Customer																BLK11-12
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Reserved for Customer																
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved for Customer																BLK14
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
																BLK31~BLK61
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
BLK31- BLK16 LOCK bit																
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
BLK15- BLK0 LOCK bit																BLK62
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
BLK61- BLK48 LOCK bit																
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
BLK47- BLK32 LOCK bit																RI K63

图 6-2 Efuse CE 使用分布

操作准备

1. 在非安全环境下使用库文件和头文件可访问 efuse;

在操作 OTP 前，应用程序需要包含 `ax_efuse_api.h` 头文件，并且应用程序需要链接 `libax_efuse.so` 库，然后就可以使用 `libax_efuse.so` 提供的 `read/write` 函数来对于 OTP 读写。

`libax_efuse.so` 库对于 OTP 的读写操作是通过 `AX_EFUSE_Write` 和 `AX_EFUSE_Read` 接口来实现，该接口每次只能写或者读一个 `block` 的内容。

2. 在安全环境下使用 `optee ca` 访问 efuse 参考 `msp/sample/optee/optee_efuse` 的 `ta` 和 `ca`。

6.1 操作过程

- 步骤1 首先初始化 OTP:

```
AX_EFUSE_Init(); 成功返回 0，失败返回错误码。
```

- 步骤2 读取 efuse 中某个 `block` 的内容:

```
AX_EFUSE_Read(blk_id, &data); 成功返回 0，失败返回错误码。
```

- 步骤3 向 OTP 中写入数据:

```
AX_EFUSE_Write(i, write_data); 成功返回 0，失败返回错误码。
```

- 步骤4 去初始化 OTP:

```
AX_EFUSE_Deinit(); 成功返回 0，失败返回错误码。
```

6.2 操作示例

可以参考 `sample_efuse.c` 和 `msp/sample/optee/optee_efuse/host/main.c`。

6.3 需要注意的问题

每个 OTP `block` 只能写一次，写入后不能修改，注意确认可以写入后再去写入 OTP 的内容。

如果需要单独写某个 `block` 的一个 `bit`，需要：

步骤1 先查看该 bit 是否为 0，如果不为 0（为 1）则无法写入。

步骤2 将该 bit 设置为 1，其他 bit 设置为 0，然后调用 `AX_EFUSE_Write` 即可。例如将 block0 的 bit5 写 1，需要调用 `AX_EFUSE_Write(0, 1 << 5)` 即可。

AEXRA CONFIDENTIAL FOR SIPEED

7 外设 DMA 操作指南

7.1 操作准备

DMA PER 有 16 个物理通道，此 DMA 主要用于外设使用，在使用 DMA 前，需要在 board 对应的 DTS 中配置使能 DMA，其所需配置在 AX620E.dtsi 中已经配置好，默认是开启的。如果需要修改，需要在对应的 DTS 文件中修改。

```
&dma_per {  
    .....  
    .....  
};
```

7.2 操作过程

此 DMA 基于 DMAENGINE 框架编程，支持 DMA_SLAVE，DMA_CYCLIC

通过 linux dmaengine 框架标准接口使用

```
dma_request_channel  
device_perp_slave_sg  
device_perp_dma_cyclic  
tx_submit  
dma_async_issue_pending
```

8 UART 操作指南

8.1 操作准备

AX620E 共支持 6 个 UART (AX620Q 仅包含 5 个 UART IP), 均为四线 UART, 在使用 UART 前, 需要在 board 对应的 DTS 中配置 UART, 所有的 UART 在 AX620E.dtsi 中已经配置好, 默认是关闭的。如果需要使用, 需要在对应的 DTS 文件中使能该 UART。

例如需要使用 UART3, 则需要在 AX620E_XXX.dts 中使能 UART3。

```
&ax_uart3 {
    status = "okay";
};
```

UART 传输支持使用 DMA(除 UART0 作为 console 外), UART 传输支持使用硬流控 (CTS/RTS), 需要配置 uart_sel 寄存器, 如下图。

寄存器地址为 0x4870000+ 0x38, 可在 Uboot 中对应的版本添加对硬流控的支持。

路径: boot/uboot/u-boot-2020.04/board/axera/

uart_ip5_sel	[11:10]	RW	2'h0	2'b00:work on RS232 2Line mode 2'b01:work on RS232 4Line mode 2'b10:work on IR 2Line mode 2'b11:work on RS485 4Line mode
uart_ip4_sel	[9:8]	RW	2'h0	2'b00:work on RS232 2Line mode 2'b01:work on RS232 4Line mode 2'b10:work on IR 2Line mode 2'b11:work on RS485 4Line mode
uart_ip3_sel	[7:6]	RW	2'h0	2'b00:work on RS232 2Line mode 2'b01:work on RS232 4Line mode 2'b10:work on IR 2Line mode 2'b11:work on RS485 4Line mode
uart_ip2_sel	[5:4]	RW	2'h0	2'b00:work on RS232 2Line mode 2'b01:work on RS232 4Line mode 2'b10:work on IR 2Line mode 2'b11:work on RS485 4Line mode
uart_ip1_sel	[3:2]	RW	2'h0	2'b00:work on RS232 2Line mode 2'b01:work on RS232 4Line mode 2'b10:work on IR 2Line mode 2'b11:work on RS485 4Line mode
uart_ip0_sel	[1:0]	RW	2'h0	2'b00:work on RS232 2Line mode 2'b01:work on RS232 4Line mode 2'b10:work on IR 2Line mode 2'b11:work on RS485 4Line mode

图 8-1 UART 模式

8.2 需要注意的问题

1. 目前芯片的 UART0 是打印 LOG 使用，建议不要将该 UART 作为其他用途，分析系统开机或者其他问题的时候可能会用到该端口作为 Debug 使用。

2. UART 支持波特率到 4000000 bits/s。

3. UART 支持小数波特率，精度计算公式如下：

整数部分 = source uart clk / (baud * 16);

小数分子部分 = (uart clk % (baud * 16)) * 16 / (baud * 16); //结果四舍五入;

小数部分 = 小数分子部分 / 16;

默认的 source uart clk 为 200M，如下为计算波特率为 115200 的精度：

$200000000 / (115200 * 16) = 108;$

$((200000000 / (115200 * 16)) * 16) / (115200 * 16) = 8$

$8 / 16 = 0.5;$

因此，当 source uart clk 为 200M 时，分频为 108.5;

$|(115200 - (200000000 / (108.5 * 16)))| / 115200 = 0.0064\%;$

3. 配置 485 模式：

```
&ax_uart3 {  
    status = "okay";  
    ax_rs485_config = <3>; //third uart  
    rs485-rtts-delay = <1 1>;  
    rs485-rx-during-tx;  
    linux,rs485-enabled-at-boot-time;  
    rs485-rtts-active-low;  
};
```

9 EMMC 操作指南

9.1 操作准备

对于使用 EMMC 的 board，只需要将保证 demo code 的 dts 中 emmc: sdhc@1B40000 相关的描述包含到对应的 dts 文件中即可。dts 中 EMMC 的描述可以理解为 AX 平台 host controller 的能力，phy 配置等信息，实际在对 EMMC device 初始化的流程中，还会获取 EMMC device 自身的能力，两者会进行匹配，最终选择合适的工作模式，该匹配的过程是由 kernel 的 MMC 框架代码保证的。

9.2 需要注意的问题

理论上，dts 中 EMMC host controller 相关的描述不需要修改，但是实际使用中可能会根据需要进行调整下支持的传输模式；

```
emmc: sdhc@1B40000 {
    compatible = "axera,sdhc";
    #address-cells = <2>;
    #size-cells = <2>;
    reg = <0x0 0x01B40000 0x0 0x10000>;
    interrupts = <GIC_SPI 9 IRQ_TYPE_LEVEL_HIGH>;
    resets = <&cpu_reset 12 0x10 1>,
            <&cpu_reset 12 0x10 1>,
            <&cpu_reset 11 0x10 1>;
    reset-names = "prst", "arst", "cardrst";
    clocks = <&cpu_clk AX620X_CLK_EMMC_EB>,
            <&cpu_clk AX620X_CLK_EMMC_CARD_EB>;
    clock-names = "aclk", "cardclk";
    status = "disabled";
};
```

SDK 中默认在 AX630C_emmc_arm64_k419.dts 中配置了 emmc device，如下：

```
&emmc{
    max-frequency = <200000000>;
    sdhci-caps-mask = <0x2 0x03200000>;
    bus-width = <8>;
```

```

no-sdio;
no-sd;
non-removable; //表示不支持热插拔
disable-wp; //disable 写保护
cap-mmc-highspeed;
mmc-hs200-1_8v;
mmc-hs400-1_8v;
mmc-hs400-enhanced-strobe;
cdns,phy-input-delay-sd-highspeed = <2>;
cdns,phy-input-delay-legacy = <4>;
cdns,phy-input-delay-sd-uhs-sdr12 = <1>;
cdns,phy-input-delay-sd-uhs-sdr25 = <2>;
cdns,phy-input-delay-sd-uhs-sdr50 = <1>;
cdns,phy-input-delay-sd-uhs-ddr50 = <2>;
cdns,phy-input-delay-mmc-legacy = <1>;
cdns,phy-input-delay-mmc-highspeed = <2>;
cdns,phy-input-delay-mmc-ddr = <2>;
cdns,phy-dll-delay-sdclk = <45>;
cdns,phy-dll-delay-sdclk-hsmmc = <23>;
cdns,phy-dll-delay-strobe = <18>;
fixed-emmc-driver-type = <4>;
status = "okay";
};

```

在 AX620E 系列芯片中支持 eMMC 的 hw reset，其中 AX630C 使用 GPIO2_A23 作为 hw reset 专用 pin，AX620Q 使用 GPIO1_A0 作为 hw reset 专用 pin，用户可根据硬件设计，配置支持 hw reset 功能。

```

&emmc{
.....
cap-mmc-hw-reset;
/* AX630C use GPIO2_A23 */
hw-reset = <&ax_gpio2 23 0>;
/* AX620Q use GPIO1_A0 */
/* hw-reset = <&ax_gpio1 0 0>; */
.....
}

```

eMMC 的稳定性测试可以选择使用 dd 命令进行测试，也可使用 FIO 或者 IOZONE 测试。

10 带宽统计操作指南

AX perf monitor 可以统计各个模块在一段时间 DDR 的访问带宽。

10.1 操作准备

在系统启动后加载 ax_perf_monitor.ko，操作命令如下：

```
#insmod /soc/ko/ax_perf_monitor.ko
```

操作完后需要卸载 ax_perf_monitor.ko：

```
# rmmod ax_perf_monitor.ko
```

10.2 操作过程

10.2.1 查看带宽

查看带宽可以通过 cat 文件节点显示：

```
# cat /proc/ax_proc/bw/bw
```

结果的显示如下：

```
/ # cat /proc/ax_proc/bw/bw
All BW:32797KB(----)
cpu/dma BW:121KB(0%), RD_BW: 105KB(0%), WR_BW: 15KB(0%)
isp BW:0KB(0%), RD_BW: 0KB(0%), WR_BW: 0KB(0%)
npu BW:0KB(0%), RD_BW: 0KB(0%), WR_BW: 0KB(0%)
mm BW:32676KB(99%), RD_BW: 32676KB(99%), WR_BW: 0KB(0%)
venc/jenc/vdec BW:0KB(0%), RD_BW: 0KB(0%), WR_BW: 0KB(0%)
```

第一行的”ALL BW: ”表示 perf monitor 检测的所有模块的带宽总和；

第二行统计的是：cpu/dma 的带宽总和，其中括号中的百分比 100%表示该通道的带宽占总的实际带宽的百分比；

第三行统计的是：isp 的带宽总和；

第四行统计的是：npu 的带宽总和；

第五行统计的是：mm 的带宽总和；

第六行统计的是：venc/jenc/vdec 的带宽总和；

BW 字段后面的数字：该 monitor 统计的所有模块的读和写的带宽之和，括号中的数字表示该通道所有模块的读和写的带宽占实际中的带宽的百分比；

RD_BW 字段后面的数字：该 monitor 统计的所有模块的读的带宽之和，括号中的数字表示该通道所有模块的读的带宽占实际中的带宽的百分比；

WR_BW 字段后面的数字：该 monitor 统计的所有模块的写的带宽之和，括号中的数字表示该通道所有模块的写的带宽占实际中的带宽的百分比；

10.2.2 设置统计周期

默认的统计周期是 1000ms，统计周期可以通过加载 ax_perf_monitor.ko 时传递参数配置，如果要设置为 500ms，方法如下：

```
# insmod /soc/ko/ax_perf_monitor.ko bw_period=500
```

10.2.3 查看统计周期

如果要查看统计的周期，可以通过 cat /proc/ax_proc/bw/bw_period 命令来查看：

```
# cat /proc/ax_proc/bw/bw_period
period:500 ms
```

10.2.4 记录带宽到文件

ax_perf_monitor.ko 驱动支持记录固定时间的带宽信息到文件中，如要记录周期为 1ms，记录的次数为 6000，则驱动加载的命令如下：

```
# insmod /soc/ko/ax_perf_monitor.ko bw_period=1 record_points=6000
# cat /proc/ax_proc/bw/bw > bw_record.xls
```

生成的 bw_record.xls 文件，用 excel 打开如下：

	A	B	C	D	E	F	G	H	I	J	K
1	index	cpu_dma_	cpu_dma_	isp_rd(KB)	isp_wr(KB)	npu_rd(KB)	npu_wr(KB)	mm_rd(KB)	mm_wr(KB)	venc_jenc_	venc_jenc_
2	456000	193	320	0	182	0	0	344	217	0	0
3	456001	1	0	0	183	0	0	477	372	0	0
4	456002	0	0	0	183	0	0	477	372	0	0
5	456003	25	4	0	183	0	0	340	254	0	0
6	456004	525	144	0	183	0	0	383	109	0	0
7	456005	281	83	0	183	0	0	34	0	1021	212
8	456006	0	0	0	183	0	0	34	0	988	203
9	456007	0	0	0	183	0	0	34	0	981	203
10	456008	0	0	0	183	0	0	25	0	1025	214
11	456009	22	6	0	183	0	0	0	0	1081	234
12	456010	0	0	0	183	0	0	25	0	1093	247
13	456011	0	0	0	183	0	0	35	0	1056	235
14	456012	0	0	0	183	0	0	34	0	958	271
15	456013	713	176	0	16	0	0	35	0	299	124
16	456014	409	83	524	430	0	0	34	0	0	0
17	456015	85	18	574	602	0	0	35	0	0	0
18	456016	1	0	549	552	0	0	34	0	0	0
19	456017	0	0	560	557	0	0	35	0	0	0
20	456018	0	0	581	580	0	0	34	0	0	0
21	456019	12	2	543	521	0	0	35	0	0	0
22	456020	0	0	570	567	0	0	34	0	0	0
23	456021	0	0	586	585	0	0	35	0	0	0
24	456022	0	0	546	521	0	0	34	0	0	0
25	456023	0	0	586	584	0	0	35	0	0	0
26	456024	147	42	593	593	0	0	34	0	0	0
27	456025	77	14	527	492	0	0	35	0	0	0
28	456026	549	186	0	31	0	0	324	249	0	0
29	456027	543	295	0	0	0	0	432	379	0	0
30	456028	10	1	0	0	0	0	452	379	0	0
31	456029	27	6	0	0	0	0	363	272	0	0
32	456030	98	23	1	48	0	0	459	313	0	0
33	456031	7	2	0	183	0	0	490	336	0	0
34	456032	0	0	0	183	0	0	488	335	0	0
35	456033	2	0	0	183	0	0	344	221	0	0
36	456034	199	80	0	183	0	0	478	372	0	0
37	456035	148	56	0	183	0	0	478	372	0	0
38	456036	735	1094	0	183	0	0	309	228	0	0
39	456037	484	124	0	183	0	0	383	109	181	24
40	456038	1	0	0	183	0	0	34	0	1034	227

表格中 `cpu_dma_rd` 和 `cpu_dma_wr` 表示 CPU/DMA 的读写带宽，`isp_wr` 表示 ISP 的写带宽。

10.3 需要注意的问题

- 注意在不使用的时候需要卸载 `ax_perf_monitor.ko`，否则该 KO 会造成系统的模块功耗增加，原因是打开带宽统计功能后，`monitor` 模块工作以及软件的开销都会造成功耗增加；
- `ax_perf_monitor` 有打开关闭的 `enable` 节点，默认处于打开状态，可通过 `echo 0 > /proc/ax_proc/bw/enable` 命令关闭，也可通过 `echo 1 > /proc/ax_proc/bw/enable` 再次打开；

- bw_period 的统计周期需要在 100ms 和 2000ms 之间（包含 100ms 和 2000ms）。

AEXRA CONFIDENTIAL FOR SIPEED

11 Audio Codec 操作指南

AX SDK 音频驱动源码结构:

表 11-1 AX SDK 音频驱动源码目录

项目	功能	路径
Sound soc	主要包含公共部分代码，包括 dapm 控制，jack、dmaengine、core 等等	sound/soc/
Axera platform	Axera 平台的 cpu dai 的驱动，比如 I2S、TDM	sound/soc/axera/
Generic platform	Simple card framework	sound/soc/generic/
Codec driver	所有的 codec driver 存放位置	sound/soc/codecs/

SDK 默认使能的是内置 codec。外置 codec 默认是不使能的，比如 es7210、es7243l、es8156、es8311 等驱动。

11.1 内置 codec 驱动操作

11.1.1 添加内置 codec driver

步骤1 添加 codec driver: sound/soc/axera/ax-actt.c。如果使用内回环时，在文件中增加 #define INTERNAL_REF 宏定义。

步骤2 修改 Makefile

```
axera_i2s-y := ax-misc.o dwc-i2s.o dwc-tdm.o ax-actt.o
```

11.1.2 修改 DTS

打开 `kernel\linux\linux-4.19.125\arch\arm64\boot\dtbs\axera` 下的 `dtb` 文件，注意开发板的板型，如果是 630Cdemo 板，则打开 `AX630C_emmc_arm64_k419.dts` 文件。（注意:这里一定是根据当前的版型以及工程类型选择对应的 `dtb`。）

步骤1 修改内置 codec 配置:

```
&audio_codec {
    gpio-mic-rp = <&ax_gpio1 7 0>; /* GPIO1_A7 */
    gpio-mic-rn = <&ax_gpio1 6 0>; /* GPIO1_A6 */
    gpio-mic-ln = <&ax_gpio1 5 0>; /* GPIO1_A5 */
    gpio-mic-lp = <&ax_gpio1 4 0>; /* GPIO1_A4 */
    gpio-pa-speaker = <&ax_gpio_ext 2 0>;
    gpio-pa-lineout = <&ax_gpio_ext 15 0>;
    status = "okay";
};
```

1. 将 `status` 的状态设置成 `okay`。
2. `gpio-mic-rp`(右声道的 P 端)、`gpio-mic-rn`(右声道的 N 端)、`gpio-mic-lp`(左声道的 P 端)、`gpio-mic-ln`(左声道的 N 端)这四个节点分别表示的是右、左 MIC 对应的 `gpio` 管脚，这里面是 `demo` 板子上的信息，客户可以根据实际板子情况修改管脚信息。
3. `gpio-pa-speaker`、`gpio-pa-lineout` 这两个节点是输出端 `speaker`、`lineout` 对应运放器的 `GPIO` 管脚（这两个管脚主要用于使能或者关闭运放器的），客户可以根据实际板子情况修改管脚信息。

步骤2 修改 `sound` 配置:

```
sound {
    status = "okay";
    compatible = "simple-audio-card";
    simple-audio-card,name = "Axera Audio";
    simple-audio-card,widgets =
        "Microphone", "Mic Jack";
    simple-audio-card,routing =
        "AMIC", "Mic Jack";
    #address-cells = <1>;
    #size-cells = <0>;
    simple-audio-card,dai-link@0 {
        format = "i2s";
        bitclock-master = <&codec_m6>;
        frame-master = <&codec_m6>;
        cpu {
            sound-dai = <&i2s_inner_slv0>;
        };
        codec_m6: codec {
            sound-dai = <&audio_codec>;
            system-clock-frequency = <12288000>;
        };
    };
    simple-audio-card,dai-link@1 {
        format = "i2s";
        cpu {
            sound-dai = <&i2s_inner_mst0>;
        };
        codec {
            sound-dai = <&audio_codec>;
            system-clock-frequency = <12288000>;
        };
    };
};
```

1. widgets 和 routing 是和内置 codec 驱动匹配的。
2. 内置 codec 需要使用的是两个 I2S (i2s_inner_mst0、i2s_inner_slv0)。
3. 节点信息请参考表格 11-1。

步骤3 修改 I2S 配置

```

i2s_inner_mst0 {
    status = "okay";
    /*i2s - pad0*/
    i2s-m-aec-cycle-sel = <0>;      /*23-22*/
    i2s-m-aec-sclk-sel = <0>;      /*21*/
    i2s-inner-codec-en = <1>;      /*20*/
    i2s-exter-codec-en = <0>;      /*19*/
    i2s-m-exter-codec-en = <0>;    /*18*/
    i2s-exter-codec-mst = <0>;     /*17*/
    i2s-m-exter-codec-mst = <0>;   /*16*/
    iis-out-tdm-en = <0>;         /*15*/
    iis-m-out-tdm-en = <0>;       /*14*/
    i2s-m-rx0-sel = <0>;          /*13*/
    i2s-m-rx1-sel = <0>;          /*12-11*/
    i2s-s-rx0-sel = <0>;          /*10-9*/
    i2s-s-rx1-sel = <2>;          /*8-7*/
    i2s-s-sclk-sel = <0>;         /*6-5*/
    tdm-m-rx-sel = <0>;           /*4-3*/
    tdm-s-rx-sel = <0>;           /*2-1*/
    tdm-s-sclk-sel = <0>;         /*0*/
};

i2s_inner_slv0 {
    status = "okay";
    /*i2s - pad0*/
    i2s-m-aec-cycle-sel = <0>;      /*23-22*/
    i2s-m-aec-sclk-sel = <0>;      /*21*/
    i2s-inner-codec-en = <1>;      /*20*/
    i2s-exter-codec-en = <0>;      /*19*/
    i2s-m-exter-codec-en = <0>;    /*18*/
    i2s-exter-codec-mst = <0>;     /*17*/
    i2s-m-exter-codec-mst = <0>;   /*16*/
    iis-out-tdm-en = <0>;         /*15*/
    iis-m-out-tdm-en = <0>;       /*14*/
    i2s-m-rx0-sel = <0>;          /*13*/
    i2s-m-rx1-sel = <0>;          /*12-11*/
    i2s-s-rx0-sel = <0>;          /*10-9*/
    i2s-s-rx1-sel = <2>;          /*8-7*/
    i2s-s-sclk-sel = <0>;         /*6-5*/
    tdm-m-rx-sel = <0>;           /*4-3*/
    tdm-s-rx-sel = <0>;           /*2-1*/
    tdm-s-sclk-sel = <0>;         /*0*/
};

```

1. SDK 中 i2s_inner_mst0、i2s_inner_slv0 的节点信息都是有保留的，客户只需要将这两个 status 设置成 okay 就行，剩下 pinmux 信息是配置好的。
2. 上面列出的信息是外回环下的配置，如果使用内回环的话则需要将 i2s-m-aec-sclk-sel 设置成 1，否则不需要修改。从原理上讲，外回环效果是会优于内回环的，所以通常建议使用外回环。
3. 节点信息请参考表格 11-2。

11.1.3 编译 axp 包

至此，修改已经完成。编译 axp，烧录至开发板，进行音频测试。

11.2 外置 codec 驱动操作

11.2.1 操作准备

如果使用外置 codec 则需要如下准备：

1. 获取 i2c 配置 codec 的 slave 地址，一般可从 codec 的芯片手册中查到。
2. 获取控制 codec 的 i2c 序号，一般可从硬件原理图中查到。
3. 获取 codec 驱动源码。

11.2.2 添加外置 codec driver

步骤 1 添加 codec driver，比如添加：sound/soc/codecs/es8311.c

步骤 2 修改 sound/soc/codecs/Kconfig

```
config SND_SOC_ES8311
    tristate "Everest Semi ES8311 2CH DAC"
    depends on I2C
```

步骤 3 修改 Makefile

```
snd-soc-es8311-objs := es8311.o

obj-$(CONFIG_SND_SOC_ES8311)+= snd-soc-es8311.o
```

步骤 4 Menuconfig 中 enable simple card 以及 codec

11.2.3 修改 DTS

打开 kernel/linux/linux-4.19.125/arch/arm64/boot/dts/axera 下的 dts 文件，注意开发板的板型，

如果是 630Cdemo 板，则打开 AX630C_emmc_arm64_k419.dts 文件。（注意:这里一定是根据当前的版型以及工程类型选择对应的 dts.）

步骤 1 修改 i2c 序号，下面是 es8311 的 i2c 配置：

```
&i2c1 {
    status = "okay";
    #address-cells = <0x1>;
    #size-cells = <0x0>;
    es8311: es8311@19 {
        compatible = "ambarella,es8311";
        reg = <0x19>;
        #sound-dai-cells = <0>;
    };
};
```

1. 如果新的 i2c 序号是 i2c2，则将上面 i2c1 节点里面的内容移至 i2c2。
2. 如果新的 codec，也可把里面的 es8311 替换成新的 codec 的名字。
3. compatible = " ambarella,es8311 ";换成 codec 驱动源码里面的 compatible。
4. reg = <0x19>;如果其他 codec 也换成新的 i2c 的 slave 地址。

步骤 2 修改 sound 配置：

```
sound {
    status = "okay";
    compatible = "simple-audio-card";
    simple-audio-card,name = "Axera Audio";
    simple-audio-card,widgets =
        "Microphone", "Mic Jack",
        "Headphone", "Headphone Jack";
    simple-audio-card,routing =
        "Headphone Jack", "DIFFERENTIAL OUT",
        "AMIC", "Mic Jack";
    #address-cells = <1>;
    #size-cells = <0>;
    simple-audio-card,dai-link@0 {
        format = "i2s";
        cpu {
            sound-dai = <&i2s_mst0>;
        };
        codec {
            sound-dai = <&es8311>;
            system-clock-frequency = <12288000>;
        };
    };
};
```

1. 如果使用其他 codec,widgets 和 routing 需要参考 codec 驱动进行修改，也可以将这个部分删除，在驱动里面直接配置也可。
2. 然后修改 codec 中 sound-dai 的名字为新的 codec 名字即可。
3. 节点信息请参考表格 11-1。

步骤 3 修改 I2S 配置

```

&i2s_mst0 {
    status = "okay";
    reg = <0x0 0x6050000 0x0 0x400>,
        <0x0 0x4870000 0x0 0x10000>;
    /*i2s - pad0*/
    i2s-m-aec-cycle-sel = <0>;          /*23-22*/
    i2s-m-aec-sclk-sel = <0>;          /*21*/
    i2s-inner-codec-en = <0>;          /*20*/
    i2s-exte-codec-en = <1>;          /*19*/
    i2s-m-exte-codec-en = <0>;        /*18*/
    i2s-exte-codec-mst = <1>;         /*17*/
    i2s-m-exte-codec-mst = <0>;       /*16*/
    iis-out-tdm-en = <0>;             /*15*/
    iis-m-out-tdm-en = <0>;          /*14*/
    i2s-m-rx0-sel = <0>;             /*13*/
    i2s-m-rx1-sel = <2>;             /*12-11*/
    i2s-s-rx0-sel = <0>;             /*10-9*/
    i2s-s-rx1-sel = <0>;            /*8-7*/
    i2s-s-sclk-sel = <0>;            /*6-5*/
    tdm-m-rx-sel = <0>;              /*4-3*/
    tdm-s-rx-sel = <0>;              /*2-1*/
    tdm-s-sclk-sel = <0>;            /*0*/
};

```

1. 使用的是其他 I2S 或者 TDM 就把其 status 设置成 okay 即可。
2. 请注意：外置 codec 可用的 I2S 和 TDM 是：i2s_mst0、i2s_slv0、i2s_tdm_mst0、i2s_tdm_slv0。
3. 节点信息请参考表格 11-2。

11.2.4 编译 axp 包

至此，修改已经完成。编译 axp，烧录至开发板，进行音频测试。

11.3 内外置 codec 混用驱动配置

如果有需求只希望使用内置的 TX 或者 RX，且搭配外置 codec 的 RX 或者 TX 使用时就需要混用配置。这里以 I2SMaster 搭配内置 codecTX、外置 codec8311RX 为例。

11.3.1 操作准备

需要如下准备：

1. 获取 i2c 配置外置 codec 的 slave 地址，一般可从 codec 的芯片手册中查到。
2. 获取控制 codec 的 i2c 序号，一般可从硬件原理图中查到。
3. 获取 codec 驱动源码。

11.3.2 添加外置 codec driver

步骤1 添加 codec driver，比如添加：sound/soc/codecs/es8311.c

步骤2 修改 sound/soc/codecs/Kconfig

```
config SND_SOC_ES8311
    tristate "Everest Semi ES8311 2CH DAC"
    depends on I2C
```

步骤3 修改 Makefile

```
snd-soc-es8311-objs := es8311.o
obj-$(CONFIG_SND_SOC_ES8311)+= snd-soc-es8311.o
```

步骤4 Menuconfig 中 enable simple card 以及 codec

11.3.3 添加内置 codec driver

步骤5 添加 codec driver：sound/soc/axera/ax-actt.c。如果使用内回环时，在文件中增加 #define INTERNAL_REF 宏定义。

步骤6 修改 Makefile

```
axera_i2s-y := ax-misc.o dwc-i2s.o dwc-tdm.o ax-actt.o
```

11.3.4 修改 DTS

打开 kernel/linux/linux-4.19.125/arch/arm64/boot/dts/axera 下的 dts 文件，注意开发板的板型，如果是 630Cdemo 板，则打开 AX630C_emmc_arm64_k419.dts 文件。（注意:这里一定是根据

当前的版型以及工程类型选择对应的 dts。)

步骤1 修改 i2c 序号，下面是 es8311 的 i2c 配置：

```
&i2c1 {
    status = "okay";
    #address-cells = <0x1>;
    #size-cells = <0x0>;
    es8311: es8311@19 {
        compatible = "ambarella,es8311";
        reg = <0x19>;
        #sound-dai-cells = <0>;
    };
};
```

1. 如果新的 i2c 序号是 i2c2，则将上面 i2c1 节点里面的内容移至 i2c2。
2. 如果新的 codec，也可把里面的 es8311 替换成新的 codec 的名字。
3. compatible = " ambarella,es8311 ";换成 codec 驱动源码里面的 compatible。
4. reg = <0x19>;如果其他 codec 也换成新的 i2c 的 slave 地址。

步骤2 修改内置 codec 配置：

```
&audio_codec {
    gpio-mic-rp = <&ax_gpio1 7 0>; /* GPIO1_A7 */
    gpio-mic-rn = <&ax_gpio1 6 0>; /* GPIO1_A6 */
    gpio-mic-ln = <&ax_gpio1 5 0>; /* GPIO1_A5 */
    gpio-mic-lp = <&ax_gpio1 4 0>; /* GPIO1_A4 */
    gpio-pa-speaker = <&ax_gpio_ext 2 0>;
    gpio-pa-lineout = <&ax_gpio_ext 15 0>;
    status = "okay";
};
```

1. 将 status 的状态设置成 okay。
2. gpio-mic-rp(右声道的 P 端)、gpio-mic-rn(右声道的 N 端)、gpio-mic-lp(左声道的 P 端)、gpio-mic-ln(左声道的 N 端)这四个节点分别表示的是右、左 MIC 对应的 gpio 管脚，这里面是 demo 板子上的信息，客户可以根据实际板子情况修改管脚信息。
3. gpio-pa-speaker、gpio-pa-lineout 这两个节点是输出端 speaker、lineout 对应运放器的 GPIO 管脚（这两个管脚主要用于使能或者关闭运放器的），客户可以根据实际板子情况修改管脚信息。

步骤3 修改 sound 配置：

```
sound {
    status = "okay";
    compatible = "simple-audio-card";
    simple-audio-card,name = "Axera Audio";
    simple-audio-card,widgets =
        "Microphone", "Mic Jack",
        "Headphone","Headphone Jack";
    simple-audio-card,routing =
        "Headphone Jack","DIFFERENTIAL OUT",
        "AMIC", "Mic Jack";
    #address-cells = <1>;
    #size-cells = <0>;
    simple-audio-card,dai-link@0 {
        format = "i2s";
        cpu {
            sound-dai = <&i2s_mst0>;
        };
        codec {
            sound-dai = <&es8311>;
            system-clock-frequency = <12288000>;
        };
    };
    simple-audio-card,dai-link@1 {
        format = "i2s";
        cpu {
            sound-dai = <&i2s_mst0>;
        };
        codec {
            sound-dai = <&audio_codec>;
            system-clock-frequency = <12288000>;
        };
    };
};
```

1. 如果使用其他 codec,widgets 和 routing 需要参考 codec 驱动进行修改,也可以将这个部分删除,在驱动里面直接配置也可。
2. 然后修改 codec 中 sound-dai 的名字为新的 codec 名字即可。
3. 节点信息请参考表格 11-1。

步骤4 修改 I2S 配置

```

&i2s_mst0 {
    status = "okay";
    /*i2s - pad0*/
    i2s-m-aec-cycle-sel = <0>;          /*23-22*/
    i2s-m-aec-sclk-sel = <0>;           /*21*/
    i2s-inner-codec-en = <1>;           /*20*/
    i2s-exter-codec-en = <1>;           /*19*/
    i2s-m-exter-codec-en = <0>;         /*18*/
    i2s-exter-codec-mst = <1>;          /*17*/
    i2s-m-exter-codec-mst = <0>;        /*16*/
    iis-out-tdm-en = <0>;               /*15*/
    iis-m-out-tdm-en = <0>;             /*14*/
    i2s-m-rx0-sel = <0>;                /*13*/
    i2s-m-rx1-sel = <0>;                /*12-11*/
    i2s-s-rx0-sel = <0>;                /*10-9*/
    i2s-s-rx1-sel = <0>;                /*8-7*/
    i2s-s-sclk-sel = <0>;               /*6-5*/
    tdm-m-rx-sel = <0>;                 /*4-3*/
    tdm-s-rx-sel = <0>;                 /*2-1*/
    tdm-s-sclk-sel = <0>;               /*0*/
};

```

1. 使用的是其他 I2S 或者 TDM 就把其 status 设置成 okay 即可。
2. 请注意：上面的配置是以外置 codec 连接在 pad0 上为前提（共有两个 pad：pad0、pad1）。同时使用内置和外置的时候 i2s-inner-codec-en 和 i2s-exter-codec-en 都需要置为 1，当外界 codec 为 salve 模式时，i2s-exter-codec-mst 置为 1。
3. 若连接的是 pad1，那么 i2s-inner-codec-en 和 i2s-m-exter-codec-en 置为 1,当外界 codec 为 salve 模式时，i2s-m-exter-codec-mst, i2s-m-rx0-sel 和 i2s-m-rx1-sel 都置为 1。
4. 节点信息请参考表格 11-2。

11.3.5 编译 axp 包

至此，修改已经完成。编译 axp，烧录至开发板，进行音频测试。

11.4 需要注意的问题

- 内置 codec 支持了录音和播放功能的同时，就不可以再支持外置 codec 了。内置 codec 当前只支持 I2S，可使用的是 i2s_inner_mst0、i2s_inner_slv0。外置 codec 可用的 I2S 和

TDM 是：i2s_mst0、i2s_slv0、i2s_tdm_mst0、i2s_tdm_slv0。

- 如果内置 codec 未能正常工作，可参考如下调试步骤：
1. 确认 codec 寄存器均能正常访问，方法：系统自带 ax_lookat 等工具，可用来访问 codec 寄存器。内置 codec 的寄存器基址是 0x23F2000。
 2. 如果 codec 寄存器均能正常访问读写，可能是 codec 寄存器配置不正确，可联系原厂进一步确认 codec 信息。
- 如果外置 codec 未能正常工作，可参考如下调试步骤：
1. 确认 codec 寄存器均能正常访问，方法：系统自带 i2cdump、i2cget、i2cset 等工具，可用来访问 codec 寄存器。
 2. 确认 IIS_MCLK、IIS_SCLK、IIS_LRCK 正确。其中：IIS_MCLK：对应 codec 的 master clock。IIS_SCLK、IIS_LRCK：i2s_mst0 控制器的输出 clock。方法：使用 tinypay/sample_audio 播放音乐，用示波器测量对应引脚，确保 clock 波形正确。
 3. 如果 codec 寄存器均能正常访问且 clock 都正确，可能是 codec 寄存器配置不正确，可参考厂商给出的 sample code 进行调试。

➤ 附表格 11-1

节点	描述
name	声卡的名称
widgets	定义音频路径中的控件，如麦克风和耳机
routing	定义控件之间的连接方式
cpu 和 codec	定义音频数据链路的两端

表格 11-1 sound 节点信息简介

➤ 附表格 11-2

bit	Register	Bit	Description
23:22	i2s_m_aec_cycle_sel	2	使用内置 codec 进行回声消除时，根据当前 i2s 分辨率进行设置，0：16bit，1：24bit，2：32bit，不使用内置回声消除通路时不需要设置
21	i2s_m_aec_sclk_sel	1	使用内置 codec 进行回声消除时，i2s master ip 的 sclk 切换，使用 soc 时钟 (i2s master) =0，使用内置 codec master 发出的时钟=1 // 使用内置的时候直接写 1
20	i2s_inner_codec_en	1	当使用内置 codec 时=1，否则=0
19	i2s_exter_codec_en	1	当 PAD_i2s 对应使用外置 codec 时=1，否则=0
18	i2s_m_exter_codec_en	1	当 PAD_i2s_m 对应使用外置 codec 时=1，否则=0
17	i2s_exter_codec_mst	1	当 PAD_i2s 对应外置 codec 为 slave 时 =1，否则=0
16	i2s_m_exter_codec_mst	1	当 PAD_i2s_m 对应外置 codec 为 slave 时 =1，否则=0
15	iis_out_tdm_en	1	PAD_i2s_sclk/lrclk 为 i2s master 的输出时=0，为 i2s_tdm master 的输出时=1
14	iis_m_out_tdm_en	1	PAD_i2s_m_sclk/lrclk 为 i2s master 的输出时=0，为 i2s_tdm master 的输出时=1
13	i2s_m_rx0_sel	1	master RX0 输入切换，输入为 PAD_i2s_sdi0 时=0，输入为 PAD_i2s_m_sdi0 时=1
12:11	i2s_m_rx1_sel	2	master RX1 输入切换，输入为 PAD_i2s_sdi1 时=0，输入为 PAD_i2s_m_sdi1 时=1，输入为 master TX sdo 时（回声消除）=2
10:09	i2s_s_rx0_sel	2	slave RX0 输入切换，输入为内置 CODEC RX 时=0，输入为 PAD_i2s_sdi0 时=1，输入为 PAD_i2s_m_sdi0 时=2
8:07	i2s_s_rx1_sel	2	slave RX1 输入切换，输入为 PAD_i2s_sdi1 时=0，输入为 PAD_i2s_m_sdi1 时=1，输入为 master sdo 同步后的数据时（回声消除）=2，输入为 slave sdo 的数据时（回声消除）=3
6:05	i2s_s_sclk_sel	2	slave sclk/lrclk 输入切换，输入为内置 CODEC RX 时=0，输入为

			PAD_i2s_sclk/lrclk 时=1, 输入为 PAD_i2s_m_sclk/lrclk 时=2
4:03	tdm_m_rx_sel	2	tdm master sdi 输入切换, 输入为 PAD_i2s_sdi0 时=0, 输入为 PAD_i2s_m_sdi0 时=1, 输入为 PAD_i2s_sdi1 时=2, 输入为 PAD_i2s_m_sdi1 时=3
2:01	tdm_s_rx_sel	2	tdm slave sdi 输入切换, 输入为 PAD_i2s_sdi0 时=0, 输入为 PAD_i2s_m_sdi0 时=1, 输入为 PAD_i2s_sdi1 时=2, 输入为 PAD_i2s_m_sdi1 时=3
0:00	tdm_s_sclk_sel	1	tdm slave sclk/lrclk 输入切换, 输入为 PAD_i2s_sclk/lrclk 时=0, 输入为 PAD_i2s_m_sclk/lrclk 时=1

表格 11-2 I2S 配置 pinmux 简介

12 USB 操作指南

12.1 操作准备

在使用 USB 之前，需要在 DTS 中配置 USB 节点参数。AX620E 支持 1 路 USB2.0 接口，支持 DRD 模式，支持 USB role 动态切换成 device 或 host。

12.2 操作过程

AX620E USB DTS 基础配置节点在 AX620E.dtsi 里，配置如下：

```
usb: axera_dwc3 {
    compatible = "axera,dwc3";
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;
    clocks = <&flash_clk AX620X_BUS_CLK_USB_EB>,
            <&flash_clk AX620X_CLK_USB_REF_EB>,
            <&flash_clk AX620X_USB_REF_ALT_CLK_EB>;
    clock-names = "bus_clk", "ref_clk", "ref_alt_clk";
    resets = <&flash_reset 25 0x14 1>,
            <&flash_reset 24 0x14 1>;
    reset-names = "reset", "preset";

    status = "disabled";

    usb2: dwc3@0x8000000 {
        compatible = "snps,dwc3";
        reg = <0x0 0x8000000 0x0 0x100000>;
        interrupts = <GIC_SPI 76 IRQ_TYPE_LEVEL_HIGH>;

        dr_mode = "peripheral";
        phy_type = "utmi";
        maximum-speed = "high-speed";
    };
};
```

定制化配置节点位于各个板子的 dts 配置文件中，例如 AX620E_emm.dts 文件中

```
/ {
    extcon_usb: extcon_usb {
        compatible = "linux,extcon-usb-gpio";
        id-gpio = <&ax_gpio0 30 0>; /* GPIO0_A30 */
        status = "okay";
    };
};
&usb {
    status = "okay";
};
&usb2 {
    dr_mode = "otg";
    extcon = <&extcon_usb>;
};
```

对于 USB DTS 节点，一般只需要配置 `dr_mode` 和 `id-gpio`。

`dr_mode` 用于设置 USB 的主从模式（host、device、DRD）。

- `dr_mode = "peripheral"`，将 USB 配置 device 模式
- `dr_mode = "host"`，将 USB 配置成 host 模式
- `dr_mode = "otg"`，将 USB 配置成 DRD 模式，类似于 OTG，支持 role 动态切换

`id-gpio` 用于配置 ID pin，例如使用 GPIO0_A30 作为 ID pin。

```
id-gpio = <&ax_gpio0 30 0>;
```

12.3 操作实例

- USB host 支持 U 盘

默认已经使能 U 盘功能，U 盘插入到 demo 板后会自动生成设备节点，挂载该节点后即可访问（读写）U 盘。

挂载方法：`mount /dev/sda1 /root/usb`

- USB host 支持鼠标或键盘

默认已经支持鼠标和键盘，可以通过 `linux input event` 来获取鼠标或键盘单击事件。

测试方法：cat /dev/input/event0 | hexdump

➤ USB device 支持虚拟串口

默认已经支持虚拟串口功能，可以通过 USB 虚拟串口来登录 console，进行 debug。

测试方法：用 Xshell 或 SecureCRT 打开 USB 虚拟串口

➤ USB 支持 UVC 功能

默认有 sample 支持 uvc 功能，代码和执行脚本以及说明文档的路径为 msp/sample/uvc，其中 README.md 为文档，uvc-gadget-composite.sh 为对应的执行脚本。

➤ USB 支持 role 切换功能

USB role 除了可以根据外接设备自动切换外，还可以在用户空间手动切换，前提是 dr_mode 被配置为 DRD 模式。

切换方法：

```
echo host > /sys/class/usb_role/80000000.dwc3-role-switch/role
```

```
echo device > /sys/class/usb_role/80000000.dwc3-role-switch /role
```

➤ USB 直出的高速信号软件配置参数动态调试及添加方法

动态调试方法：

通过 ax_lookat 工具对相关寄存器进行动态配置调试。

举例：USB 调试配置命令

```
ax_lookat 0x10030048 -s 0x1900
```

说明：USB 对应的配置寄存器地址是 0x10030048，0x1900 说明将寄存器的 11 和 12 位置 1，对应寄存器就是将 hstx_deemp 设置为 11。

配置寄存器信息如下，

Field Name	Bit	Type	Reset Value
Reserved	[31:18]	RO	14'h0
GLB_utmi_suspendm_en	[17]	RW	1'h0
GLB_utmi_suspendm	[16]	RW	1'h0
GLB_ls_en	[15]	RW	1'h0
GLB_ex_pd_func	[14]	RW	1'h0
GLB_hstx_vcc09v_mode	[13]	RW	1'h0
GLB_hstx_deemp	[12:11]	RW	2'h0
GLB_debug_sel	[10:7]	RW	4'h0
GLB_hs_bist_mode	[6]	RW	1'h0
GLB_lpm_alive	[5]	RW	1'h0
GLB_pll_en	[4]	RW	1'h0
GLB_vcontrol	[3:0]	RW	4'h0

添加方法：

调试完成后可以将最终的配置参数信息添加到/linux-5.15.73/drivers/usb/dwc3/dwc3-axera.c 文件的 dwc3_of_simple_probe 函数中 reset 函数之后，添加方法如下

```
#define AXERA_CLK_AND_RST_EN

+#define FLASH_SYS_USB2_PHY_GLB1 0x10030048
+
+ /* clock and reset */
+ #define FLASH_CLK_RST_BASE_ADDR 0x10030000
+ .....
static int dwc3_of_simple_probe(struct platform_device *pdev)
{
    struct device_node *np = dev->of_node;

    int ret;
+ void iomem *addr;

    dev_info(dev, "axera usb probe\n");
    .....
+ addr = ioremap(FLASH_SYS_USB2_PHY_GLB1, 4);
+ value |= 0x1900;
+ writel(value, addr);
+ iounmap(addr);
+
    pm_runtime_set_active(dev);
```

➤ USB 接口的 WiFi 功能

目前 SDK 支持 RTL8188FTV 型号的 WiFi，支持 STA 模式和 AP 模式，使用方法如下：

1) 代码路径

驱动路径： kernel/osdrv/third_drv/rtl8188fu_wifi

固件路径: rootfs_extend/lib/firmware/rtlwifi/rtl8188fufw.bin

运行脚本: rootfs/opt/scripts/usb-hotspot.sh 和 rootfs/opt/scripts/usb-wifi.sh

2) 编译

```
cd kernel/osdrv/third_drv
```

```
make p=AX620E_emmc all install
```

3) 调试方法

STA 模式: usb-wifi.sh start wifissid wifipwd

AP 模式: usb-hotspot.sh start wifissid wifipwd

4) 测试

STA 模式: ping -I wlan0 www.baidu.com

AP 模式: 手机搜索对应热点连接。

12.4 需要注意的问题

- demo 板 USB 接口默认被配置成 device 模式，对应配置节点是 usb 支持 role 的动态切换，需修改对应的配置。
- USB device 在系统启动后会被配置为虚拟串口，可以使用它来登录 terminal，进行 debug，使用方法同串口一致，默认用户名是 root，密码是 123456。
- USB 支持 8 对双向 endpoint。
- USB 不支持 OTG 协议，但是支持 DRD，也能够实现 role 的动态切换。
- U-boot 支持 USB Host 2.0 功能，用于实现 USB U 盘升级功能。

13 EMAC 操作指南

13.1 操作准备

AX620E 最大支持 1 路千兆网卡，支持 RGMII 和 RMII 接口，芯片内部集成了一个 RMII 接口的百兆 EPHY 芯片。demo 板中默认使能千兆 RGMII 外置网卡，采用的是 JLSemi JL2101 PHY 芯片，如果需要使用别的 PHY 芯片，DTS 需要做相应修改。

13.2 操作过程

demo 板 EMAC DTS 节点在 AX620E.dtsi 配置如下：

```
eth0: ethernet@104C0000{
    compatible = "axera,dwmac-4.10a";
    reg = <0x0 0x104C0000 0x0 0x10000>;
    clocks = <&flash_clk AX620X_ACLK_EMAC_EB>,
            <&flash_clk AX620X_EPHY_CLK_EB>,
            <&flash_clk AX620X_CLK_EMAC_RGMII_TX_EB>,
            <&flash_clk AX620X_CLK_EMAC_RMII_PHY_EB>,
            <&flash_clk AX620X_CLK_EMAC_PTP_REF_EB>;
    clock-names = "emac_aclk", "ephy_clk", "rgmii_tx_clk",
                  "rmii_phy_clk", "ptp_clk";
    resets = <&flash_reset 8 0x14 1>,
            <&flash_reset 9 0x14 1>,
            <&flash_reset 0 0x20 1>;
    reset-names = "emac_rst", "ephy_rst", "ephy_shutdown";
    interrupt-parent = <&gic>;
    interrupts = <GIC_SPI 80 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "macirq", "eth_wake_irq", "eth_lpi";
    rx-fifo-depth = <32768>;
    tx-fifo-depth = <16384>;
    snps,tso;
    status = "disabled";
};
```

对应各种定制化的板子配置如 AX620E_emmc.dts，RGMII 1000M 外置网卡配置如下：

```

&eth0 {
    status = "okay";

    phy-handle = <&phy0>;
    phy-mode = "rgmii";
    phy-rst-gpio = <&ax_gpio1 27 0>; /* GPIO1_A27 */

    mdio {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy0: jl2xxx-phy@1 {
            compatible = "ethernet-phy-id937c.4030";
            jl2xxx,led-enable = <JL2XXX_LED_CTRL_EN>;
            jl2xxx,led-mode = <JL2XXX_CFG_LED_MODE>;
            jl2xxx,led-period = <JL2XXX_GLOBAL_PERIOD_MS>;
            jl2xxx,led-on = <JL2XXX_GLOBAL_ON_MS>;
            jl2xxx,led-polarity = <JL2XXX_LED_POLARITY>;
            jl2xxx,patch-enable = <JL2XXX_PATCH_CTRL_EN>;
            jl2xxx,rgmii-enable = <JL2XXX_RGMII_CTRL_EN>;
            jl2xxx,rgmii-tx-delay = <JL2XXX_RGMII_TX_DLY_2NS>;
            jl2xxx,rgmii-rx-delay = <JL2XXX_RGMII_RX_DLY_2NS>;
            jl2xxx,clk-enable = <JL2XXX_CLK_CTRL_EN>;
            jl2xxx,flr-enable = <JL2XXX_FLR_CTRL_EN>;
            jl2xxx,flr-delay = <JL2XXX_FLR_DELAY>;
            jl2xxx,wol-enable = <JL2XXX_WOL_CTRL_EN>;
            jl2xxx,interrupt-enable = <JL2XXX_INTR_CTRL_EN>;
            jl2xxx,downshift-enable = <JL2XXX_DSFT_CTRL_EN>;
            jl2xxx,downshift-count = <JL2XXX_DSFT_AN_CNT>;
            jl2xxx,work_mode-enable = <JL2XXX_WORK_MODE_CTRL_EN>;
            jl2xxx,work_mode-mode = <JL2XXX_WOEK_MODE_MODE>;
            jl2xxx,lpbk-enable = <JL2XXX_LPBK_CTRL_EN>;
            jl2xxx,lpbk-mode = <JL2XXX_LPBK_MODE>;
            jl2xxx,slew_rate-enable = <JL2XXX_SLEW_RATE_CTRL_EN>;
            jl2xxx,rx_out-enable = <JL2XXX_RXC_OUT_CTRL_EN>;
            reg = <1>;
        };
    };
};

```

RMII 100M 内置 EPHY 网卡配置如下：

```
&eth0 {
    status = "okay";

    phy-handle = <&phy0>;
    phy-mode = "rmii";
    axera-ephy,led0-mode = <EPHY_LED_SPEED>;
    axera-ephy,led0-enable;
    axera-ephy,led1-mode = <EPHY_LED_LINK>;
    axera-ephy,led1-enable;
    axera-ephy,led-polarity = <EPHY_LED_LOW_ACTIVE>;
    mdio {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy0: axera-ephy@1 {
            compatible = "ethernet-phy-id0044.1400";
            reg = <1>;
        };
    };
};
```

demo 板中默认使能千兆 RGMII 网卡，如果用户想切换内置网卡只需要将 AX620E_emmc.dts 节点里的 RMII EPHY 网卡节点使能，并禁用千兆 RGMII 网卡节点，编译更新 DTS。

一般来说，EMAC DTS 节点保持默认配置即可。但如果更换 PHY 芯片，需要做如下修改：

步骤1 确定 MII 接口类型，并更新至 DTS。例如采用 RMII 接口：

```
phy-mode = "rmii";
```

步骤2 确定 PHY reset gpio，并更新至 DTS。例如采用 GPIO1_A27 来复位 PHY：

```
phy-rst-gpio = <&ax_gpio1 27 0>;
```

步骤3 修改 PHY compatible 参数。例如采用 RTL8201 100M PHY：

```
compatible = "ethernet-phy-id001c.c816";
```

001c: PHY 的 16 位 ID 寄存器 1 值，也就是 OUI 的 bit3~18，16 进制格式。
c816: PHY 的 16 位 ID 寄存器 2 值，也就是 OUI 的 bit19~24，16 进制格式。

步骤4 确定 PHY 地址，例如 PHY 地址配置成 2：

```
phy0: phy@2 {
    compatible = "ethernet-phy-id001c.c816";
    reg = <2>;
};
```

步骤5 配置或编写相应的 PHY 驱动。kernel 已经支持大部分常见的以太网 PHY，选择对应厂商，使能即可。

```
Device Drivers --->
  [*] Network device support --->
    *- PHY Device support and infrastructure --->
      *- MDIO Bus/PHY emulation with fixed speed/link PHYs
        <*> Marvell PHYs
        <*> Realtek PHYs
        .....
```

AXERA_EPHY: CONFIG_AXERA_EPHY

JLSEMI PHYs: CONFIG_JLSEMI_PHY

Marvell PHYs: CONFIG_MARVELL_PHY

Realtek PHYs: CONFIG_REALTEK_PHY

13.3 需要注意的问题

- EMAC 驱动一般不需要做二次开发，即使更换了 PHY 芯片，也只需适配 PHY 芯片驱动，EMAC 和 PHY 在 kernel 中已经通过 MDIO 接口解耦合。
- 默认已经打开 TSO 功能，如需关闭，在 DTS 中去掉 snps, tso; 即可。
- 更换 PHY 芯片型号后，如果需要在 uboot 中使用网络功能，也需要做相应的适配。
 - 确定 MII 接口类型和 PHY reset gpio，并更新至 uboot DTS eth0 node。例如采用 RMII 接口：

```
phy-mode = "rmii";
phy-rst-gpio = <&port2a 28 0>;
```

- 配置或编写相应的 PHY 驱动。uboot 已经支持大部分常见的以太网 PHY，选择对应

厂商，使能即可。

```
Device Drivers --->
  *- Ethernet PHY (physical media interface) support --->
    [*] Marvell Ethernet PHYs support    //marvell.c
    [*] Realtek Ethernet PHYs support
    .....
```

- Kernel 启动后默认采用 DHCP 自动获得 IP 地址，如需改成静态 IP，需要修改 `/etc/init.d/net_config`，重启后不会失效。

```
# eth0
ipaddr0=192.168.1.2
netdev0=eth0
netmask0=255.255.255.0
gateway0=
```

- AX620E EMMC 和 NAND 工程默认使用外置千兆网卡，NOR 工程默认使用内置百兆 EPHY 网卡，修改默认配置方法参考 13.2 操作过程。
- AX620E 也支持外置 RMII PHY 网卡，网卡端除了以上配置外，需要在 `dtb` 中额外添加 `axera,out_rmii` 标识来区别内置网卡。
- AX620E 内部 EPHY 网卡 LED 已支持 DTS 配置，`led0/1-mode` 表示该 LED 的模式支持 3 种模式 LINK 模式、DUPLEX 模式、SPEED 模式，`led0/1-enabe` 表示该 LED 被分配为网卡 LED 使用，如果不使用该 LED 需要删除该配置。`led-polarity` 指示 LED 极性有效与硬件设计相关，芯片默认是高电平有效。
- AX620E EMMC 工程设置静态 IP 方式修改配置 `/etc/init.d/net_config`，Nor 工程里该分区无法写入，需要将该文件拷贝到 `/customer` 分区下修改配置生效。

```
#cp /etc/init.d/net_config /customer/net_config
```

修改配置 `/customer/net_config` 文件，拔插网线或重启系统生效。

```
#!/bin/sh

# configure network

# getting ip mode
# dhcp: default, get ip from server
# static: static IP, we need to configure following variable
#
# eth0
ipaddr0=192.168.1.100
netdev0=eth0
netmask0=
gateway0=
```

AEXRA CONFIDENTIAL FOR S

14 I2C 操作指南

14.1 操作准备

AX620E 系列中 AX630C SOC 一共有 8 个 I2C，其中 8 个都是高速，I2C 从 I2C0 到 I2C7，AX620Q 仅包含 4 个高速 I2C master，从 I2C0 到 I2C3，他们从两组 PAD 出 pin，使用同一个 IP。

14.2 操作准备

14.2.1 配置 pinmux

将相应 pin 脚的功能切换至 I2C 功能。

```
0x0230400C, 0x00000083, /* PadName = I2C0_SCL      Fuction = I2C0_SCL */
0x02304018, 0x00000083, /* PadName = I2C0_SDA      Fuction = I2C0_SDA */
0x02304024, 0x00000083, /* PadName = I2C1_SCL      Fuction = I2C1_SCL */
0x02304030, 0x00000083, /* PadName = I2C1_SDA      Fuction = I2C1_SDA */
```

14.2.2 控制器驱动在 kernel 中的命令

1. i2c 命令:

用于传输的 i2cget、i2cset、i2cdump 仅能支持 8byte 寄存器地址，也就是一个地址周期的器件，想读写 16byte 地址的器件必须使用 i2ctransfer;

- (1) 显示可用的 I2C 总线：i2cdetect -l;
- (2) 显示 I2C0 号总线上所有可用设备：i2cdetect -r -y 0;
- (3) 显示 I2C0 号总线上 0x1b 设备的寄存器值。MODE 为 byte：i2cdump -f -y 0 0x1b b;
- (4) 显示 I2C0 号总线上 0x1b 设备 0x0 和 0x1 寄存器地址，按字节读取：i2cget -f -y 0 0x1b 0x0;
- (5) 在 I2C1 号总线上 0x2d 七位地址，写值 0x42 到一个 8 位寄存器 0x11 的 I2C 设备：

```
i2cset 1 0x2d 0x11 0x42;
```

(6) 从 i2c4 号总线 0x38 设备的 0x3a01 寄存器开始读 16 个字节的数据, w2: 表示寄存器 0x3a01 的长度为 2 个字节: `i2ctransfer -y -f 4 w2@0x38 0x3a 0x01 r16;`

(7) 从 i2c4 号总线, 向 0x32 写入 2 字节数据, 然后读取 6 个字节的数据: `i2ctransfer 4 w2@0x32 0x00 0x01 r6;`

14.2.3 内核设备树的配置

在设备树配置文件 AX620E.dtsi 中有如下配置:

```
aliases {
    i2c0 = &i2c0;
    i2c1 = &i2c1;
    i2c2 = &i2c2;
    i2c3 = &i2c3;
    i2c4 = &i2c4;
    i2c5 = &i2c5;
    i2c6 = &i2c6;
    i2c7 = &i2c7;
}
```

I2C0 的设备树配置: 对应的是 1 个快速率(NS)的 I2C:

```
i2c0: i2c@4850000 {
    compatible = "snps,designware-i2c";
    reg = <0x0 0x4850000 0x0 0x400>;
    interrupts = <GIC_SPI 135 IRQ_TYPE_LEVEL_HIGH>;
    scl-gpio = <&ax_gpio0 24 0>;
    sda-gpio = <&ax_gpio0 25 0>;
    pinctrl-names = "default", "gpio";
    pinctrl-0 = <&i2c0_scl_pins>, <&i2c0_sda_pins>;
    pinctrl-1 = <&i2c0_scl_gpio0_a24_pins>, <&i2c0_sda_gpio0_a25_pins>;
    clock-frequency = <400000>;
    i2c-sda-hold-time-ns = <500>;
    clocks = <&periph_clk AX620X_PCLK_I2C_MST0_EB>,
            <&periph_clk AX620X_CLK_I2C_MST0_EB>;
    clock-names = "pclk", "clk";
    resets = <&periph_reset 18 0x18 1>,
            <&periph_reset 19 0x18 1>;
}
```

```

        reset-names = "prst","rst";
        status = "disabled";
    };

```

注：其中红色标识部分需要根据 i2c mode 修改，如下

```

Standard-mode:
    clock-frequency = <100000>;
    i2c-sda-hold-time-ns = <2000>;
    i2c-scl-falling-time-ns = <300>;
    i2c-sda-falling-time-ns = <1000>;
Fast-mode:
    clock-frequency = <400000>;
    i2c-sda-hold-time-ns = <500>;
Fast-mode Plus:
    clock-frequency = <1000000>;
    i2c-sda-hold-time-ns = <50>;
    i2c-scl-falling-time-ns = <120>;
    i2c-sda-falling-time-ns = <120>;
HighSpeed-mode: /*此配置 8 个高速 I2C 支持*/
    clock-frequency = <3400000>;
    i2c-scl-falling-time-ns = <30>;
    i2c-sda-falling-time-ns = <50>;
    i2c-sda-hold-time-ns = <50>;

```

14.3 I2C Recovery

在某些特定的场景下，SDA 线会被从机拉低拉死，IP 端支持 recovery 机制来解决此类问题，在设备树中配置了 sda 和 scl 管脚如下：（pinctrl 相关配置请参阅 [pinctrl 章节](#)）

```

/* PAD i2c0 */
&i2c0 {
    status = "okay";
    scl-gpio = <&port2a 21 0>;
    sda-gpio = <&port2a 22 0>;
    pinctrl-names = "default", "gpio";
    pinctrl-0 = <&i2c0_scl_pins>, <&i2c0_sda_pins>;
    pinctrl-1 = <&gpio2_a21_pins>, <&gpio2_a22_pins>;
};

```

！ 注意： 在快启项目中（fastnor，fastemmc），riscv 先占用 i2c，使用完成后再释放 i2c，此时 linux 开始有控制权，在 riscv 使用中，linux 调用 i2c 读写会返回 busy，因此 sensor 需要单独占用一个 i2c，外设不要使用此 i2c，防止 linux 中外设驱动 probe 流程中部分外设需要使用 i2c 读取外设 id 时，probe 流程失败。

AEXRA CONFIDENTIAL FOR SIPEED

15 RTC 操作指南

15.1 操作准备

在使用 RTC 前，需要配置 dts 使能 rtc，AX620E 基本的 RTC 计时和 alarm 功能。

```
axera_rtc_pwr:rtc-top@23E0000 {
    compatible = "axera,axi-top-rtc";
    reg-num = <2>;
    reg = <0x0 0x23E0000 0x0 0x1000>,
        <0x0 0x2340000 0x0 0x1000>;
    interrupts = <GIC_SPI 107 IRQ_TYPE_LEVEL_HIGH>;
    status = "disabled";
};

&axera_rtc_pwr {
    status = "okay";
};
```

15.2 时间同步与 NTP

Rtc 时间查看和配置查阅 hwclock 命令相关资料，Rtc 和 ntp 时间的配置查看对应脚本 /etc/init.d/ S50crontabs

```
* * * * * /usr/sbin/ntpd -n -q -p cn.pool.ntp.org
* * * * * hwclock -w -f /dev/rtc0
```

15.3 rtc sample 测试

在 sdk 中有 rtc_sample.c 是用来测试 rtc 的测试程序，sample 中的 alarm 时间为 15s 应该不是所有的场景都适合，读者可以自行修改代码设置时间。

板端直接运行 sample_rtc 即可，也可以指定 rtc 设备，如：sample_rtc /dev/rtc0。

15.4 需要注意的问题

目前 rtc 的 alarm 的精度为 2.5s 左右，如果小于 2.5s，可能不产生对应的 alarm 中断。

16 PWM 外设操作指南

16.1 操作准备

16.1.1 AX pwm 介绍

从芯片的 pinlist 看，理论上最多支持 PWM0 ~PWM2 总共 3 组，每组 4 路，共计 12 个 pwm source，编号 PWM00-PWM03，PWM10-PWM13，PWM20-PWM23，但是需要根据硬件设计确认使用哪个 pwm source。

说明：

AX pwm 的输入 clk 最大 24Mhz，因此理论上 pwm 输出的频率必须小于 24Mhz。

pwm 编号有好几种，对应关系如下表

软件编号	pinlist 里编号
pwm00	pwm00
pwm01	pwm01
pwm02	pwm02
pwm03	pwm03
pwm10	pwm04
pwm11	pwm05
pwm12	pwm06
pwm13	pwm07
pwm20	pwm08

pwm21	pwm09
pwm22	pwm10
pwm23	pwm11

pinlist 中带_M 的引脚与不带_M 的引脚对应如下,以 PWM07 输出为例, 输出引脚可以选择 PWM07 或者 PWM7_M,但是只能二者择一, 不能同时选择。

Pinlist 里带_M 编号	Pinlist 里不带_M 的编号
PWM0_M	PWM00
PWM1_M	PWM01
PWM2_M	PWM02
PWM3_M	PWM03
PWM4_M	PWM04
PWM5_M	PWM05
PWM6_M	PWM06
PWM7_M	PWM07

16.1.2 配置 pinmux

使用 pinmux 工具, 根据硬件设计配置使用的 pwm pin 脚, 选择对应的 function 等, 可参考 [1 pinmux 操作指南](#)。

16.1.3 配置 u-boot

目前默认 u-boot-2020.04 中 pwm 驱动和 dts 被编译到 uboot 镜像中。如有改动, 需要确认编

译到 dts 中的 pwm 描述以及 pwm driver code。

Check 点:

1. [u-boot-2020.04/arch/arm/dts/ax620e.dtsi](#) 中有 pwm0~pwm2 描述信息。
2. [u-boot-2020.04/configs/AX630C_emmc_arm64_k515_defconfig](#) 中 CONFIG_DM_PWM=y 和 CONFIG_PWM_AXERA=y。
3. u-boot-2020.04/drivers/pwm 目录下 Kconfig、Makefile 以及 axera_pwm.c。
4. 可在 [u-boot-2020.04/configs/AX630C_emmc_arm64_k515_defconfig](#) 中配置 CONFIG_CMD_PWM=y

来添加 pwm 测试的 uboot 命令

16.1.4 配置 kernel

如果只在 u-boot 中使用 pwm 功能，则 kernel 中不用关注，如果是 linux kernel 起来后使用，需要关注。

目前默认 kernel 中 pwm 驱动和带有 pwm 描述的 dts 被编译到 kernel 镜像中。如有改动，需要确认编译到 dts 中 pwm 描述和 pwm driver code。

Check 点:

1. [linux-5.15.73/arch/arm64/boot/dts/axera/AX620E.dtsi](#) 中有 pwm0~pwm2 描述信息。
2. [linux-5.15.73/arch/arm64/configs/axera_AX630C_emmc_arm64_k515_defconfig](#) 中 CONFIG_PWM=y, CONFIG_PWM_SYSFS=y 和 CONFIG_PWM_AXERA=y。
3. linux-5.15.73/drivers/pwm 目录下 Kconfig、Makefile、pwm-axera.c。

16.2 操作实例

16.2.1 pinmux 配置

pwm 不管是 u-boot 中使用，还是 linux 起来后使用，都需要根据硬件设计，对使用的 pwmsource 对应的 pin 进行配置。

以 PWM00 举例，参考 [1 pinmux 操作指南](#)，更新 xxx_pinmux.h 文件。

```
.....
0x04250254, 0x00010003, /* PadName = DVS_PWM0          Fuction = PWM00*/
.....
```

注意，如果使用某一路 PWM 来作为 vddcore 调压，为防止系统 sleep 时出现问题，pwm 驱动里做了相应处理，会在 pwm suspend 函数里将 pin 的 pwm 功能切换为普通 gpio 功能，需要根据硬件实际情况，修改设备树如下节点

```
pwm2: pwm2@6062000 {
    compatible = "axera,ax620e-pwm";
    reg = <0x0 0x6062000 0x0 0x400>,
        <0x0 0x4870000 0x0 0x1000>;
    #pwm-cells = <2>;
    resets = <&periph_reset 29 0x1c 1>,
        <&periph_reset 25 0x1c 1>,
        <&periph_reset 26 0x1c 1>,
        <&periph_reset 27 0x1c 1>,
        <&periph_reset 28 0x1c 1>;
    reset-names = "pwm-rst", "pwm-ch0-rst",
        "pwm-ch1-rst", "pwm-ch2-rst",
        "pwm-ch3-rst";
    #if 0
    clocks = <&periph_clk AX620X_PCLK_PWM2_EB>,
        <&periph_clk AX620X_CLK_PWM20_EB>,
        <&periph_clk AX620X_CLK_PWM21_EB>,
        <&periph_clk AX620X_CLK_PWM22_EB>,
        <&periph_clk AX620X_CLK_PWM23_EB>;
    clock-names = "pclk", "pwm-ch0-clk",
        "pwm-ch1-clk", "pwm-ch2-clk",
        "pwm-ch3-clk";
    #endif
    clk-sel-addr-offset = <0x0>;
    clk-sel-offset = <0xD>;
    clk-glb-eb-addr-offset = <0x4>;
    clk-glb-eb-offset = <0x9>;
    clk-eb-addr-offset = <0x8>;
    clk-eb-offset = <0x1B 0x1C 0x1D 0x1E>;
    clk-p-eb-addr-offset = <0x10>;
    clk-p-eb-offset = <0x1>;
    pin_sleep_setting = <0x2301024 0x60043>;
    status = "disabled";
};
```

0x2301024 是 pin 对应的 pinmux 寄存器，0x60043 是进 sleep 后需要配置的 pin 状态

16.2.2 u-boot 中使用

u-boot 中操作 pwm 的接口参见 u-boot-2020.04/include/pwm.h。

```
int pwm_set_config(struct udevice *dev, uint channel, uint period_ns, uint
duty_ns);
int pwm_set_enable(struct udevice *dev, uint channel, bool enable);
```

参数说明：

dev: uboot 中每组 pwm 创建一个 udevice, 对应 PWM 的组号 (0-2)
 channel: 组内的 channel 号, 对应 PWM 中的组内号 (0-3)
 period_ns: 该 channel 的 pwm 周期, 单位 ns
 duty_ns: 占空比参数, 一个 pwm 周期内高电平的时间, 单位 ns

使用举例方法 1: 操作 PWM00, pwm 周期 240Khz, 占空比: 50%。

```
uclass_get_device(UCLASS_PWM, 0, &dev);    //获取第 0 组 pwm 的 udevice
pwm_set_config(dev, 0, 4167, 2084);        //配置第 0 组, channel 0 的 period 和
duty 值
pwm_set_enable(dev, 0, true);              //使能该 channel0 pwm
```

使用举例方法 2: 同样以 PWM00 为例。

步骤1 在 device 的 dts 描述中添加如下信息:

```
A_Device {
.....
pwms = <&pwm0 0 4167>;    //参数分别为 pwm0, channel 0, period
.....
};
```

步骤2 在 device 的驱动中参考如下 code:

```
struct udevice *pwm = NULL;
struct ofnode_phandle_args args;

dev_read_phandle_with_args(dev, "pwms", "#pwm-cells", 0, 0, &args);
uclass_get_device_by_ofnode(UCLASS_PWM, args.node, &pwm);
//获取 pwm 组的 udevice 指针, 从 A_Device dts 中获取到的 channel、period 值保存在
//args.args[0]、args.args[1]中

pwm_set_config(pwm, args.args[0], args.args[1], 2084);
//配置对应 pwm 组, channel 的 period, duty 值, 单位 ns

pwm_set_enable(pwm, args.args[0], 1); //使能该 channel 输出
```

此外, uboot 中还提供 pwm 命令, 以使用户能够方便的操作 PWM, 可在 [u-boot-2020.04/configs/AX630C_emmc_arm64_k515_defconfig](#) 中配置 CONFIG_CMD_PWM=y, 然后 pwm -h 可查看该命令的使用方法, 根据提示操作需要的 pwm 即可。

16.2.3 应用层使用

使用前请参考 16.1.4 配置 kernel 一节，确保 kernel 中已经包含 pwm 驱动。应用层可以通过 sysfs 节点来操作 pwm，以 PWM00 举例：

```
echo 0 > /sys/class/pwm/pwmchip0/export    //pwmchip0 下生成 pwm0 目录

echo 4167 > /sys/class/pwm/pwmchip0/pwm0/period    //设置第 0 组，channel0 period 值

echo 2084 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle // 设置 channel0 duty 值

echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable      //使能该 channel
echo 0 > /sys/class/pwm/pwmchip0/pwm0/enable      //关掉该 channel
```

说明：

- 所有的 pwm 组目录都生成在/sys/class/pwm 目录。pwmchipX，代表一个组，kernel 的 pwm 框架中对所有 pwm channel 统一编码，从 0 开始计数，此 X 代表该 pwm 组的 pwm channel 在统一编码框架中的起始编号，但是 pwmchipX 内部的 pwm channel 编码是从 0 开始的。请参考表 16-1。
- 应用层可通过文件操作 API，如 fopen、fread、fwrite 来操作相关的 sysfs 节点。
- 注意上述举例操作 sysfs 节点有操作顺序。

表16-1 pwm sys 节点说明

pwm 组	组目录 sys 节点	组内 channel sys 节点
0	/sys/class/pwm/pwmchip0	/sys/class/pwm/pwmchip0/pwm0~3
1	/sys/class/pwm/pwmchip4	/sys/class/pwm/pwmchip4/pwm0~3
2	/sys/class/pwm/pwmchip8	/sys/class/pwm/pwmchip8/pwm0~3

16.2.4 kernel 中使用

kernel 中开发模块驱动中，如需要使用 pwm，以 PWM00 为例，使用方法如下：

步骤1 设备的 dts 中加入需要使用 pwm source 的描述。

```
A_Device {
```

```
.....  
pwms = <&pwm0 0 4167>;    //参数分别为 pwm0, channel, period  
.....  
};
```

步骤2 设备的 driver code 中使用 pwm 接口操作 pwm。

```
pwm_dev = of_pwm_get(pdev->dev.of_node, NULL);    //保存 struct  
pwm_device 指针  
  
pwm_config(pwm_dev, 2084, 4167);    //调用 config 接口配置 duty、period, 单  
位 ns  
pwm_enable(pwm_dev);                //使能 pwm 输出
```

17 Thermal 操作指南

17.1 操作准备

AX620E 支持内部的 thermal，thermal 支持 2 个温度点的中断和一个温度点的重启，目前在 kernel 内只使用了最高温度点的重启功能，其他两个点的中断没有使用（中断后只是打印一下 log，没有做一些 cooling 的处理），thermal 目前读取的是通道 0 的温度。

Thermal 的 dts 配置请参考：

kernel/linux/linux-x.xx.xx/arch/arm64/boot/dts/axera/AX620E.dtsi 文件中的描述，其中 threshold 表示低温设置为 80 度，在低于 80 度时系统进入中断处理函数，target 表示中温设置为 105 度，在高于 105 度时系统进入中断处理函数，soc_crit 表示高温设置为 120 度，表示在 120 度的时候系统就会重启。

```

tsensor: tsensor@2000000 {
    compatible = "axera,ax620e-tsensor";
    reg = <0x0 0x2000000 0x0 0x100000>;
    resets = <&comm_reset 18 0x54 1>,
           <&comm_reset 19 0x54 1>;
    reset-names = "thm_prst", "thm_rst";
    clocks = <&common_clk AX620X_CLK_THM_EB>,
           <&common_clk AX620X_CLK_THM_EB>;
    clock-names = "clk_thm", "pclk_thm";
    interrupt-controller;
    #interrupt-cells = <2>;
    interrupts = <GIC_SPI 61 IRQ_TYPE_LEVEL_HIGH>;
    status = "okay";
    #thermal-sensor-cells = <1>;
};

thermal-zones {
    soc_thm: soc_thm {
        polling-delay = <1000>;
        polling-delay-passive = <100>;
        sustainable-power = <4500>;
        /* sensor ID */
        thermal-sensors = <&tsensor 0>;
        trips {
            threshold: trip-point@0 {
                temperature = <80>;
                hysteresis = <0>;
                type = "passive";
            };
            target: trip-point@1 {
                temperature = <105>;
                hysteresis = <0>;
                type = "passive";
            };
            soc_crit: soc-crit {
                temperature = <120>;
                hysteresis = <0>;
                type = "passive";
            };
        };
    };
};

```

图17-1 Thermal 的 DTS 配置参考

如果需要针对不同的 board 配置不同的 thermal 温度的话，可以在对应的 board 的 dts 中重新定义一下 thermal-zones 就可以了。

```
tsensor: tsensor@2000000 {
    compatible = "axera,ax620e-tsensor";
    reg = <0x0 0x2000000 0x0 0x100000>;
    resets = <&comm_reset 18 0x54 1>,
           <&comm_reset 19 0x54 1>;
    reset-names = "thm_prst", "thm_rst";
    clocks = <&common_clk AX620X_CLK_THM_EB>,
           <&common_clk AX620X_CLK_THM_EB>;
    clock-names = "clk_thm", "pclk_thm";
    interrupt-controller;
    #interrupt-cells = <2>;
    interrupts = <GIC_SPI 61 IRQ_TYPE_LEVEL_HIGH>;
    status = "okay";
    #thermal-sensor-cells = <1>;
};

thermal-zones {
    soc_thm: soc_thm {
        polling-delay = <1000>;
        polling-delay-passive = <100>;
        sustainable-power = <4500>;
        /* sensor ID */
        thermal-sensors = <&tsensor 0>;
        trips {
            threshold: trip-point@0 {
                temperature = <80>;
                hysteresis = <0>;
                type = "passive";
            };
            target: trip-point@1 {
                temperature = <105>;
                hysteresis = <0>;
                type = "passive";
            };
            soc_crit: soc-crit {
                temperature = <120>;
                hysteresis = <0>;
                type = "passive";
            };
        };
    };
};
```

图17-2 Thermal board DTS 配置参考

17.2 查看当前温度

系统开机后，可以在 sys 文件系统中查看系统的当前温度的命令为：

```
cat /sys/class/thermal/thermal_zone0/temp
```

18 NPU 信息操作指南

在`/proc/ax_proc/npu`节点下可以查看当前 NPU 利用率和版本信息，以及模型耗时情况。

18.1 NPU 利用率

在运行过一次 NPU 后，执行下面步骤：

步骤1 使能 npu 利用率统计功能：

```
Echo 1 >/proc/ax_proc/npu/enable
```

步骤2 通过读取`/proc/ax_proc/npu/top_period_ms`文件中的数值，获取采样周期，默认值为 1000ms，同时支持修改操作：

```
Cat /proc/ax_proc/npu/top_period_ms  
Echo 2000 > /proc/npu/top_period_ms
```

步骤3 获取当前 NPU 利用率：

```
cat /proc/ax_proc/npu/top
```

```
/proc/ax_proc/npu # cat top  
core:vnpu-Non  
time:12  
period:1000000  
utilization:39
```

The diagram shows the output of the 'cat top' command with four green arrows pointing from labels in boxes to specific fields in the output:

- Arrow from 'npu核类型' to 'core:vnpu-Non'
- Arrow from '采样次数' to 'time:12'
- Arrow from '采样周期' to 'period:1000000'
- Arrow from 'NPU利用率' to 'utilization:39'

图 18-1 top 命令执行结果

18.2 NPU 模型耗时信息

步骤1 打开 npu 模型运行时间统计功能，或者关闭该功能

```
Echo 1 >/proc/ax_proc/npu/enable
Echo 0 >/proc/ax_proc/npu/enable
```

步骤2 运行 NPU 模型，然后执行下面命令，可获取最近执行的 100 Task 的耗时详情，输出结果的时间单位：微秒（us）

```
cat /proc/ax_proc/npu/top_detail
```

task_id	start time	waiting-time	runing-time	model-name/handle
43	5160530642	727	346	npu_model_param/29
44	51605308490	19	66	async_model/22
45	51605308502	105	356	async_model/19
46	51605308936	64	360	npu_model_param/23
47	51605309243	148	337	async_model/24
48	51605311758	25	336	async_model/16
49	51605312437	14	327	async_model/15
50	51605318288	22	357	async_model/14
51	51605322195	23	4045	async_model/5
52	51605322207	4148	593	async_model/6
53	51605322966	4013	374	async_model/7

图 18-2 top_detail 的结果

表 18-1 top_detail 各个字段意义

字段	说明	示例
Start_time	提交 npu task 的起始时间点（us）	123456 us
Waiting-time	等待执行的耗时（us）	12 us
Runing-time	在 npu 上真实运行的耗时（us）	34 us
Model_name/handle	当前 npu 模型名字/句柄	Model_name

！ 注意：

该命令需要在 NPU 运行过程中执行。如果 cat 命令之后没有上面的输出结果，或者没有任何内容输出，请确认当前 NPU 是否还在继续执行。

步骤3 执行下面命令可以获取 NPU 运行概况信息

```
cat /proc/ax_proc/npu/top_sum
```

```

| 1. Buffered Tasks in NPU |
|-----|
[---core 0 ---:
[model / task buffered : 0   runned : 0]
[cv   / task buffered : 0   runned : 0]

| 2. Max/Avg/Min value of Model Running Period |
|-----|
[model_handle : ax_model_NULL/0
 run_time [max : 19689   avg : 12345   min : 11625] us

| 3. Number of Handle for Model |
|-----|
[Models handles | used : 0   not-used : 200 ]

| 4. Number of Handle for CV |
|-----|
[CV handles     | used : 1   not-used : 7 ]

| 5. Number of Task ID |
|-----|
[tasks handle   | used : 0   not-used : 300 ]

```

在npu core上未执行和已执行的task 数量值

某一个模型执行时间耗时统计包含最大值/平均值/最小值

已使用和未使用的模型句柄数量值

已使用和未使用CV句柄的数量值

已使用和未使用的Task ID的数量值

图 18-3 top_sum 的结果

！ 注意：

该命令需要在 NPU 正在运行过程中执行。如果 cat 命令之后没有上面的输出结果，或者没有任何内容输出，请确认当前 NPU 是否还在继续执行。

18.3 NPU 版本信息

开机后，执行：

```
cat /proc/ax_proc/npu/version
```

19 UID 操作指南

AX UID 是用来表示每一个 AX620E 系列芯片的唯一 ID，用户可以通过读取该 ID 来区分不同的 AX620E 芯片，该 UID 是一个 64bit 的二进制数字，存放在 Efuse 中的 block[1-0]中。

19.1 操作准备

读取 AX UID 的方法是进入 AX620E 系列芯片系统的控制台，执行 `cat` 命令即可显示该芯片的 UID。

```
# cat /proc/ax_proc/uid  
ax_uid: 0x000000007bf65c17
```

19.2 需要注意的问题

- `ax_uid` 是基于 `ax_hwinfo.c` 来实现的，所以它依赖于内核配置宏 `CONFIG_AX_HWINFO`，默认使能了该宏，如果该宏配置被删除，则 `/proc/ax_proc/uid` 文件会不存在；
- 如果是工程芯片，该文件可能读取的为全 0，UID 没有在芯片工厂写入。

20 SD/SDIO 操作指南

20.1 操作准备

1. 对于使用 sdio 的 board，需要保证 sdio: sdhc@104D0000 相关的设备描述包含到对应的 dts 文件中。对于使用 sd 的 board，需要保证 sd: sdhc@104E0000 相关的设备描述包含到对应的 dts 文件中。
2. sd host controller 的 driver，demo code 中默认编译进 kernel，如有改动，需要确认编译到对应的 driver code。

☞ Check 点：

- 确保 drivers\mmc\host\sdhci-axera.c 被编译
- 确保 config 中配置 CONFIG_MMC_SDHCI_AXERA=y

3. 对于 sdio 设备，如 WiFi，需要客户自己开发或者获取第三方 sdio 外设的 driver code。
4. AX630C 有一路 sd 和一路 sdio；AX620Q 只有一路 sd，没有 sdio 接口。

20.2 操作过程

20.2.1 SD 卡

针对诸如 micro sd 存储卡，只需要使能 dts sd 节点、硬件上有卡座，插卡即可使用，可通过诸如 fdisk、mkdosfs 等命令来对 sd 卡进行分区、格式化等操作，然后通过 mount 命令挂载即可对 sd 卡进行读写操作。

DTS 配置

理论上，dts 中 sd host controller 相关的描述不需要修改，但是实际使用中可能会根据需求调整下支持的传输模式等配置；

dtsi:

```
sd: sdhc@104E0000 {
```

```

compatible = "axera,sdhc";
#address-cells = <2>;
#size-cells = <2>;
reg = <0x0 0x104E0000 0x0 0x10000>;
interrupts = <GIC_SPI 74 IRQ_TYPE_LEVEL_HIGH>;
resets = <&flash_reset 17 0x14 1>,
        <&flash_reset 16 0x14 1>,
        <&flash_reset 15 0x14 1>;
reset-names = "prst", "arst", "cardrst";
clocks = <&flash_clk AX620X_ACLK_SD_M_EB>,
        <&flash_clk AX620X_PCLK_SD_M_EB>,
        <&flash_clk AX620X_CLK_SD_CARD_EB>;
clocks = <&sysclk>, <&sysclk>, <&sysclk>;
clock-names = "aclk", "pclk", "cardclk";
status = "disabled";
};

```

SDK 中 dts 默认配置了 sd device, 如下:

AX620Q:

```

&sd{
    sdhci-caps-mask = <0x7 0x00200000>;
    bus-width = <4>;
    max-frequency = <200000000>;
    cap-sd-highspeed;
    no-sdio;
    no-mmc;
    disable-wp;
    #if 1
        broken-cd;
    #else
        cd-gpios = <&ax_gpio1 11 0>;
        cd-inverted;
        cd-debounce-delay-ms = <300>;
    #endif

    cdns,phy-input-delay-sd-highspeed = <2>;
    cdns,phy-input-delay-legacy = <3>;
    cdns,phy-input-delay-sd-uhs-sdr12 = <3>;
    cdns,phy-input-delay-sd-uhs-sdr25 = <2>;
}

```

```
cdns,phy-input-delay-sd-uhs-sdr50 = <1>;
cdns,phy-input-delay-sd-uhs-ddr50 = <1>;
cdns,phy-dll-delay-sdclk = <0>;

status = "okay";
};
```

AX630C:

```
&sd{
    sdhci-caps-mask = <0x7 0x00200000>;
    bus-width = <4>;
    max-frequency = <200000000>;
    cap-sd-highspeed;
    sd-uhs-sdr12;
    sd-uhs-sdr25;
    sd-uhs-sdr50;
    sd-uhs-ddr50;
    sd-uhs-sdr104;
    no-sdio;
    no-mmc;
    disable-wp;
#if 1
    // poll
    broken-cd;
#else
    //gpio detect
    cd-gpios = <&ax_gpio1 11 0>;
    cd-inverted;
    cd-debounce-delay-ms = <300>;
#endif

    cdns,phy-input-delay-sd-highspeed = <2>;
    cdns,phy-input-delay-legacy = <3>;
    cdns,phy-input-delay-sd-uhs-sdr12 = <3>;
    cdns,phy-input-delay-sd-uhs-sdr25 = <2>;
    cdns,phy-input-delay-sd-uhs-sdr50 = <1>;
    cdns,phy-input-delay-sd-uhs-ddr50 = <1>;
    cdns,phy-dll-delay-sdclk = <0>;

    status = "okay";
```

```
};
```

20.2.2 SDIO 接口

DTS 配置

AX620E.dtsi 中 sdio host controller 相关的配置不需要修改：

```
sdio: sdhc@104D0000 {
    compatible = "axera,sdhc";
    #address-cells = <2>;
    #size-cells = <2>;
    reg = <0x0 0x104D0000 0x0 0x10000>;
    interrupts = <GIC_SPI 73 IRQ_TYPE_LEVEL_HIGH>;
    resets = <&flash_reset 20 0x14 1>,
            <&flash_reset 19 0x14 1>,
            <&flash_reset 18 0x14 1>;
    reset-names = "prst", "arst", "cardrst";
    clocks = <&flash_clk AX620X_ACLK_SDIO_M_EB>,
            <&flash_clk AX620X_PCLK_SDIO_M_EB>,
            <&flash_clk AX620X_CLK_SDIO_M_CARD_EB>;
    clock-names = "aclk", "pclk", "cardclk";
    status = "disabled";
};
```

SDK 的 dts 中配置了 sdio device，默认是 disabled 状态。根据需要调整传输模式，频率等配置：

```
&sdio{
    sdhci-caps-mask = <0x7 0x00200000>;
    bus-width = <4>;
    max-frequency = <200000000>;
    cap-sd-highspeed;
    sd-uhs-sdr25;
    sd-uhs-sdr12;
    sd-uhs-sdr50;
    sd-uhs-ddr50;
    //sd-uhs-sdr104;
    no-sd;
    no-mmc;
    disable-wp;
    vol-sw-gpio = <&ax_gpio0 17 0>;
```

```
cdns,phy-input-delay-sd-highspeed = <2>;
cdns,phy-input-delay-legacy = <3>;
cdns,phy-input-delay-sd-uhs-sdr12 = <3>;
cdns,phy-input-delay-sd-uhs-sdr25 = <2>;
cdns,phy-input-delay-sd-uhs-sdr50 = <0>;
cdns,phy-input-delay-sd-uhs-ddr50 = <0>;
cdns,phy-dll-delay-sdclk = <0>;

status = "disabled";
};
```

如上的注释部分，对 sdio 设备，注释掉 no-sdio，使能 no-sd、no-mmc 属性，其他的属性根据具体硬件设计来做相应配置。

SDIO driver code 适配

对于 sdio 设备，一般会调用 sdio_register_driver 接口来完成 sdio 外设 driver 的注册，需要保证在调用 sdio_register_driver 前完成 sdio 设备的识别。

建议在 sdio 外设 driver code 中，在上电 code 后，调用 mmc_detect_change 触发对 sdio device 的 rescan 操作来完成 sdio 设备的识别动作，默认 SDK 中增加了 rescan 接口供驱动层使用。

```
void axera_sdio_rescan(void)
{
    if (!sdio_host) {
        pr_err("axera sdio_host is NULL\n");
        return;
    }

    pr_debug("axera sdio_host rescan begin\n");

    mmc_detect_change(sdio_host, 0);
}

EXPORT_SYMBOL_GPL(axera_sdio_rescan);
```

20.2.3 SDIO WiFi 外设

目前 SDK 已经支持 AP6256 SDIO WiFi，代码路径：sdk/kernel/osdrv/third_drv/ap6256_wifi

使用方法如下：

1) 编译

```
cd ~/mc20e-sdk/kernel/osdrv/third_drv/ap6256_wifi
```

```
make p=xxx all install
```

将会在 ap6256_wifi 下生成 bcmhdhd.ko

2) 安装

拷贝 ap6256_wifi 目录下的 bcmhdhd.ko、fw_bcm43456c5_ag.bin、nvram_ap6256.txt 到板端

3) 创建/etc/wpa_supplicant_tplink.conf

4) 加载驱动

```
insmod ./bcmhdhd.ko "firmware_path=./fw_bcm43456c5_ag.bin"
```

```
"nvram_path=./nvram_ap6256.txt"
```

```
ifconfig wlan0 up
```

```
wpa_supplicant -i wlan0 -Dnl80211 -c /etc/wpa_supplicant_tplink.conf &
```

5) 测试

```
udhcpc -i wlan0
```

```
ping
```

6) 注意事项

- 目前 demo 板不支持直接安装 SDIO WiFi 模块，需要单独修改或设计电路
- 需要配置 WL_REG_ON 和 WL_WAKE_AP

在 dhd_gpio.c dhd_wlan_init_gpio()中，修改 gpio_wl_reg_on 和 gpio_wl_host_wake，且 WL_REG_ON pinmux 配置悬空，WL_WAKE_AP pinmux 配置下拉

```
269 | {  
270 | |     gpio_wl_reg_on = 1; //WL_REG_ON ... GPIO0_A1  
271 | #ifdef CUSTOMER_OOB  
272 | |     gpio_wl_host_wake = 0; //WL_WAKE_HOST ... GPIO0_A0  
273 | #endif  
274 | }
```

- 如果要支持 UHS1 模式（sdr104 不支持），需要在 sdio dts 节点配置 IO voltage gpio
- 暂时不支持 sdr104 模式

20.3 需要注意的问题

- 1) SD 仅支持轮询方式或 GPIO 中断方式检测卡，如果使用轮询方式，在 dts 中加入 broken-cd; 如果使用 GPIO 中断方式，在 dts 中加入 cd-gpios = <&ax_gpio0 2 0>;
- 2) AX630C SD 接口支持 UHS1、HS、DS mode; AX620Q SD 接口仅支持 HS、DS mode，不支持 UHS1
- 3) 对于 AX620Q，如果不使用 SD 卡，也可以将 SD 接口当作 sdio 接口使用，配置相应的 dts 即可。
- 4) dts 中 sd host controller 相关的描述实际使用中可能会根据用途等进行修改，例如需要调整下支持的传输模式，比如需要支持到 sdr50 模式，只需注释掉 sd-uhs-sdr104;，其他一些常见的修改见注释部分。

```
&sdio {  
    .....  
    bus-width = <4>;  
    .....  
    cap-sd-highspeed;  
    sd-uhs-sdr12;  
    sd-uhs-sdr25;  
    sd-uhs-sdr50;  
    sd-uhs-sdr104;    //以上几行是支持的速度模式  
    .....  
};
```

21 ADC 操作指南

21.1 操作准备

AX620E 芯片内部集成了一个 ADC，一共有4个通道，可以使用它来测量外部的电压，例如通过在 ADC 外接不同的上拉/下拉电阻，通过 ADC 通道0读取电压做为 board ID 来使用。

21.2 操作过程

步骤1 UBOOT 下 ADC 调试命令：

```
AXERA-UBOOT=> adc list
```

结果如下：

```
- adc@0x2000000
```

```
AXERA-UBOOT=>adc single adc@0x2000000 0
```

结果如下（adc single adc@0x2000000 channel（channel 为0到3））：

```
187
```

步骤2 2.LINUX 下 ADC 调试命令：

```
cd /sys/bus/iio/devices/iio\:device0/
```

```
cat in_voltage0_raw/ in_voltage1_raw/ in_voltage2_raw/ in_voltage3_raw;
```

3.UBOOT 下 ADC 计算外部电压的操作方法：

1) DAT 和 voltage 的对应关系， $voltage = DATA / 1024 * VREF(1.8V)$

2) 如果读取 chan0/1/2/3/4，需要读取0x20000A0，0x20000A4，0x20000A8，0x20000AC，（uboot 参考下面接口）；

uboot 接口：

```
int read_adc_value(void)
{
    unsigned int val;
    int channel = 0;
```

```
int ret;

ret = adc_channel_single_shot("adc@0x2000000", channel, &val);
if (ret)
    return ret;

return val;
}
```

AEXRA CONFIDENTIAL FOR SIPEED

22 APB TIMER 操作指南

22.1 操作准备

在使用 apb timer 前，需要在 board 对应的 DTS 中配置 apb_timer

```
apb_timer1: dw_apb_timer@6030000 {
    compatible = "axera,dw-apb-timer";
    reg = <0x0 0x6030000 0x0 0x400>;
    resets = <&periph_reset 6 0x20 1>,
            <&periph_reset 7 0x20 1>;
    reset-names = "preset", "reset";
    clocks = <&periph_clk AX620X_PCLK_TIMER1_EB>,
            <&periph_clk AX620X_CLK_TIMER1_EB>;
    clock-names = "pclk", "clk";
    interrupts = <GIC_SPI 128 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 129 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 130 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 131 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "channel0", "channel1",
                     "channel2", "channel3";
    timer_index = <1>;
    status = "disabled";
};
```

22.2 操作过程

接口 1 打开并申请 timer:

Return: 成功: timer handle 失败: NULL

dev_id: 要操作的 timer, 0 或者 1

channel_id: 要操作那个 channel, 编号 0~3

```
u32 *timer_open(unsigned int dev_id, unsigned int channel_id);
```

接口 2 下发 timer 配置:

timer_handle: 打开并申请的 timer handle

usec: 要配置的时间 us

handle: 回调函数, 工作在进程上下文

arg: 回调函数的参数

return: 0: 成功 -1: 失败

```
int timer_set(u32 *timer_handle, u32 usec, void (*handle)(void *), void *arg);
```

接口 3 关闭 timer:

timer_handle: 打开并申请的 timer handle

```
void timer_close(u32 *timer_handle);
```

22.3 需要注意的问题

使用 apb timer 需先申请并打开 timer, 使用完毕后需要关闭 timer 否则会导致其他人无法只用该 timer。Apb_Timer0 已在 SDK 内部使用, 不能在做普通定时器使用

23 PINCTRL 操作指南

23.1 操作准备

soc 内部管脚众多，而多数管脚又可以配置成不同的功能，我们称之为管脚复用；可根据 pinmux 表格配置项目所需的 function。

pinctrl 并不仅仅是设置 function，还要配置引脚，比如上拉(bias-pull-up)、下拉(bias-pull-down)、不拉(bias-disable)、驱动能力(drive-strength)、施密特触发器(input-schmitt-enable)等等。我们在 AX620E kernel/linux/linux-5.15.48/drivers/pinctrl/axera/pinctrl-ax620e.c 包含了所有 pad 的 function；通过在 kernel/linux/linux-5.15.48/arch/arm64/boot/dts/axera/AX620E_xxx.dts 引用可以正常使用对应 function；在 kernel/linux/linux-5.15.48/arch/arm64/boot/dts/axera/AX620E_pinctrl.dtsi 包含所有 function 对应的 mux 和 config，在引用 pinctrl 前，需要在 kernel/linux/linux-5.15.48/arch/arm64/boot/dts/axera/AX620E_pinctrl.dtsi 中找到对应的 function pins。

以 I2C0_SDA func 为例：

(1) 在 kernel/linux/linux-5.15.48/arch/arm64/boot/dts/axera/AX620E_pinctrl.dtsi 查找

```
i2c0_sda_pins: i2c0_sda {
    mux {
        pins = "I2C0_SDA"; /* PAD NAME */
        function = "I2C0_SDA"; /* FUNCTION NAME */
    };
    configs {
        pins = "I2C0_SDA"; /* PAD NAME */
        drive-strength = <3>; /* AX620Ex 驱动能力 为0 ~ 15
(bit0~bit3) 值越大驱动能力越强 */
        input-schmitt-enable = <0>; /* 施密特触发器 为 0 或者 1
(bit4) */
        bias-pull-up; /* Bias-pull-down (bit6 为 1) 下拉 ,bias-
pull-up (bit7 为 1) 上拉, bias-disable (bit6~bit7 为 0) 不拉*/
    };
};
```

(2) 在 kernel/linux/linux-5.15.48/arch/arm64/boot/dts/axera/AX620E_xxx.dts 引用，以 i2c1 为

例：

```
&i2c0 {
    pinctrl-names = "default", "gpio"; /*与 pinctrl-num 对应, default 不需要调用
接口也会被配置, pinctrl 子系统会自动配置 default 状态的 config 和 funtion_sel, 其他状态必
须要调用切换接口*/
    pinctrl-0 = <&i2c0_scl_pins>, <&i2c0_sda_pins>; /*对应 pinctrl-names 的
default*/
    pinctrl-1 = <&i2c0_scl_gpio0_a24_pins>, <&i2c0_sda_gpio0_a25_pins>; /*
对应 pinctrl-names 的 gpio*/
}
```

23.2 操作过程

步骤1 切换对应状态(pinctrl 默认三步设置所选 function):

```
struct pinctrl *pinctrldemo;
struct pinctrl_state *s;

pinctrldemo = devm_pinctrl_get(dev); //获取当前设备的 pinctrl 句柄
if (IS_ERR(pinctrldemo)) {
    ret = PTR_ERR(pinctrldemo);
    pr_err("Cannot find pinctrlaud!\n");
    return;
}
s = pinctrl_lookup_state(pinctrldemo, "gpio"); //获取需要设置的 state
if (IS_ERR(s)) {
    ret = PTR_ERR(s);
    pr_err("Cannot find pinctrllookup_state!\n");
    return;
}
ret = pinctrl_select_state(pinctrldemo, s); //设置为对应的 state
pinctrl_put(pinctrldemo); //释放 pin
```

步骤2 切换到 default 状态 (pinctrl 默认接口可一步切换为 default 状态):

```
struct pinctrl *pinctrldemo;
pinctrldemo = devm_pinctrl_get_select_default(dev); //一步设置成 pinctrl-
0 (default) 的功能
pinctrl_put(pinctrldemo); //释放 pin
```

步骤3 一步切换到对应状态（pinctrl 默认接口可一步切换为所选 function 状态）：

```
struct pinctrl *pinctrldemo;
pinctrldemo=devm_pinctrl_get_select(dev, "gpio");//一步设置成 pinctrl-1
gpio 的功能了
pinctrl_put(pinctrldemo); //释放 pin
```

步骤4 PINCTRL 兼容 GPIO 接口：

```
1.设置 gpio 方向输入
pinctrl_gpio_request(gpio_num_store) ;//gpio_num_store 代表 gpio 编号
pinctrl_gpio_direction_input(gpio_num_store);
pinctrl_gpio_free(gpio_num_store);
1.设置 gpio 方向输出
pinctrl_gpio_request(gpio_num_store) ;//gpio_num_store 代表 gpio 编号
pinctrl_gpio_direction_output(gpio_num_store);
pinctrl_gpio_free(gpio_num_store);
```

23.3 需要注意的问题

1. 使用 pinctrl 接口必须 free，不然同 pin 其他 function 切换不了。

普通 GPIO 切换使用

(1) PINMUX:

i2c0_sda	I2C0_SDA	SPI_M2_CS2	RISC_TRSTN	PWM01	CLK_AUX0_1	UART2_RTS	GPIO0_A25	ANALOG_TEST13
----------	----------	------------	------------	-------	------------	-----------	-----------	---------------

(2) kernel/linux/linux-5.15.48/arch/arm64/boot/dts/axera/AX620E_pinctrl.dtsi:

```
i2c0_sda_pins: i2c0_sda {
    mux {
        pins = "I2C0_SDA";
        function = "I2C0_SDA";
    };
    configs {
        pins = "I2C0_SDA";
        drive-strength = <3>;
        input-schmitt-enable = <0>;
        bias-pull-up;
    };
};
```

```
i2c0_sda_gpio0_a25_pins: i2c0_sda_gpio0_a25 {
    mux {
        pins = "I2C0_SDA";
        function = "GPIO0_A25";
    };
    configs {
        pins = "I2C0_SDA";
        drive-strength = <3>;
        input-schmitt-enable = <0>;
    };
};
```

(3) .kernel/linux/linux-5.15.48/arch/arm64/boot/dts/axera/AX620E_emmc.dts

```
&i2c0 {
    pinctrl-names = "default", "gpio";
    pinctrl-0 = <&i2c0_scl_pins>, <&i2c0_sda_pins>;
    pinctrl-1 = <&i2c0_scl_gpio0_a24_pins>, <&i2c0_sda_gpio0_a25_pins>;
};
```

(4) 在驱动中切换操作

方法一:

```
struct pinctrl *pinctrldemo;
struct pinctrl_state *s;

pinctrldemo= devm_pinctrl_get(dev); //获取当前设备的 pinctrl 句柄
if (IS_ERR(pinctrldemo)) {
    ret = PTR_ERR(pinctrldemo);
    pr_err("Cannot find pinctrlaud!\n");
    return;
}
s = pinctrl_lookup_state(pinctrldemo, "gpio "); //获取需要设置的 state
if (IS_ERR(s )) {
    ret = PTR_ERR(s);
    pr_err("Cannot find pinctrllookup_state!\n");
    return;
}
ret = pinctrl_select_state(pinctrldemo, s); //设置为对应的 state
pinctrl_put(pinctrldemo); //释放 pin
```

方法二:

```
struct pinctrl *pinctrldemo;  
pinctrldemo=devm_pinctrl_get_select(dev, "gpio");//一步设置成pinctrl-1的功能了  
pinctrl_put(pinctrldemo); //释放 pin
```

切换回 default 可以用方法一 二也可用下面接口：

```
struct pinctrl *pinctrldemo;  
pinctrldemo=devm_pinctrl_get_select_default(dev);//一步设置成pinctrl-0  
(default) 的功能  
pinctrl_put(pinctrldemo); //释放 pin
```

(5) 当切换为 **gpio func** 后可以直接调用 **gpio** 驱动操作该 **pin**。

24 DDR 带宽限制操作指南

24.1 操作准备

各个需要带宽限制功能的驱动模块会在相应 device 设备初始化之后调用 BW limiter 注册接口函数，其结果会在 `/proc/ax_proc/bw_limit/` 下生成与各个驱动模块对应的节点，如下面图例：

```
1 /proc/ax_proc/bw_limit # ls
2 cpu2ddr      isp_itp      mm_dpu_lite  mm_tdp      register    vpu_venc
3 flash_axdma  isp_yuv      mm_gdc      mm_vpp      vpu_jenc
4 isp_ife      mm_dpu      mm_ive      npu         vpu_vdec
```

24.2 操作过程

成功创建各个带宽限制功能的 proc 节点后，可以使用 echo 命令来分别设置相应的读写带宽限制值，以其中 dma 带宽限制功能为例，800 表示 800MB，1000 表示 1000MB（1GB）

```
echo 800 > /proc/ax_proc/bw_limit/flash_axdma/limiter_val_rd
echo 1000 > /proc/ax_proc/bw_limit/flash_axdma/limiter_val_wr
```

当需要关闭带宽限制功能的时候，可以 echo 0 到相应 proc 节点，如下示例：

```
echo 0 > /proc/ax_proc/bw_limit/flash_axdma/limiter_val_xxx
```

24.3 需要注意的问题

1. 执行 echo 命令需要在各个 device 设备初始化之后。
2. npu 的带宽限制只能限制读写的总带宽，不能单独限制读或者写的带宽，所以 npu 目录下只有一个节点 `limiter_val_sum_rdwr`，使用方式同上。

25 I2C SLV 操作指南

25.1 操作准备

AX620E SOC 一共有 2 个 I2C SLV。以内核 i2c-slave-eeeprom.c 测试代码为例：

```
&i2c_slv0 {
    status = "okay";
    #address-cells = <0x1>;
    #size-cells = <0x0>;
    i2c_slave_eeeprom: i2c_slave_eeeprom@55 {
        compatible = "axera,slave-24c02";
        reg = <0x40000055>;
        status = "okay";
    };
};
&i2c_slv1 {
    status = "disabled";
};
```

如上，从机地址需要把 bit30 位置 1，从机地址为 0x55 时，reg 属性需要填写 0x40000055

25.2 操作示例

可以参考 linux-5.15.73(linux-4.19.125)/drivers/i2c/i2c-slave-eeeprom.c。

25.3 需要注意的问题

i2c slave 不能区分地址和值，若需要进行地址，值的通信方式时，只能单字节传输，不能连续传输。